LABORATORY 08 – Data Link Layer and Application Layer

AUTORS

Andrés Felipe Chavarro Plazas

David Santiago Espinosa Rojas

TEACHER

Diego Alejandro Murcia Cespedes

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

VIGILADA MINEDUCACIÓN

COMPUTER NETWORKS

GROUP 01

Bogota, Colombia

May 11, 2025

OBJETIVE

Revisar y configurar la infraestructura LAN de una empresa, trabajando con redes Ethernet y WiFi, dispositivos de interconexión, y servicios de la capa de aplicación como servidores web, DNS, correo electrónico y bases de datos.

MATERIALS & TOOLS

- Laboratory computers with internet access.
- Virtualization software.
- Switches
- Wireshark & Packet tracer
- Wires type cat

INTRODUCTION

In this lab, we will continue developing a technological infrastructure, integrating wired and wireless workstations, physical and virtual servers, and network devices such as switches and routers. We will use tools such as Wireshark and Cisco Packet Tracer to analyze the behavior of Ethernet and Wi-Fi networks, as well as configure essential services at the application layer. We will also explore advanced configurations such as VLANs, secure wireless networks, and the implementation of dynamic web services with databases, thus strengthening our practical understanding of the data link and application layers.

THEORETICAL FRAMEWORK

Network hardware management is essential for the proper operation of any communications infrastructure. Devices such as switches, routers, wireless access points, and physical or virtual servers form the foundation upon which local area networks and their connection to external networks are built. Layer 2 and Layer 3 switches allow multiple devices to be interconnected in a network, facilitating segmentation and logical addressing of data. Routers, for their part, enable communication between different networks using routing protocols and IP address assignment.

In this context, understanding the basic configuration of these devices is crucial. This includes assigning names, setting access passwords, configuring interfaces, defining VLANs to segment the network, and applying security mechanisms such as access control lists and authentication methods in Wi-Fi networks such as WPA2-PSK.

Network analysis, on the other hand, allows for diagnosing network behavior and ensuring its proper functioning. Tools like Wireshark allow you to capture and examine network packets in real time, displaying details such as MAC addresses, protocols used, frame content, and possible transmission errors. This type of analysis is useful for verifying connectivity, understanding the operation of protocols such as ARP, DHCP, and TCP/IP, and detecting potential failures or bottlenecks.

Furthermore, the use of simulation environments such as Cisco Packet Tracer provides a safe space to design, test, and visualize network operations without the need for physical hardware. With these tools, it is possible to observe how devices build their routing or switching tables, how the spanning tree algorithm works to avoid loops, and how packets behave in networks with VLANs or wireless connections.

Together, mastering network hardware and analysis tools strengthens professionals' ability to build efficient, secure networks tailored to an organization's needs.

SETUP

In this section, the lab's wired and wireless networks were configured, including switches, PCs, and routers. Connectivity was verified, Ethernet frames were analyzed with Wireshark, VLANs were implemented to segment the network, and secure Wi-Fi networks were configured.

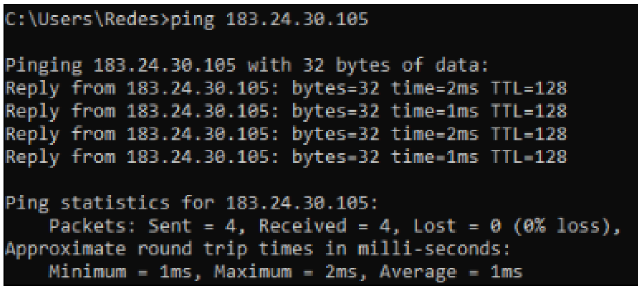
BASIC SWITCH CONFIGURATION

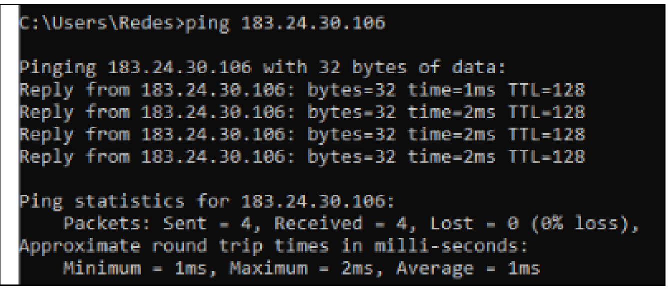
Just like in the previous lab, we configured the switch assigned to us by the instructor during the lab. To do this, we accessed the PuTTY console and entered the necessary commands to connect the ports on our table and its computers to the designated IP ranges.

```
Switch1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch1(config)#interface fastEthernet0/10
Switch1(config-if)#no shutdown
Switch1(config-if)#no ip domain lookup
Switch1(config)#interface fastEthernet0/3
Switch1(config-if)#no shutdown
Switch1(config-if)#no ip domain lookup
Switch1(config)#exit
Switch1#
```

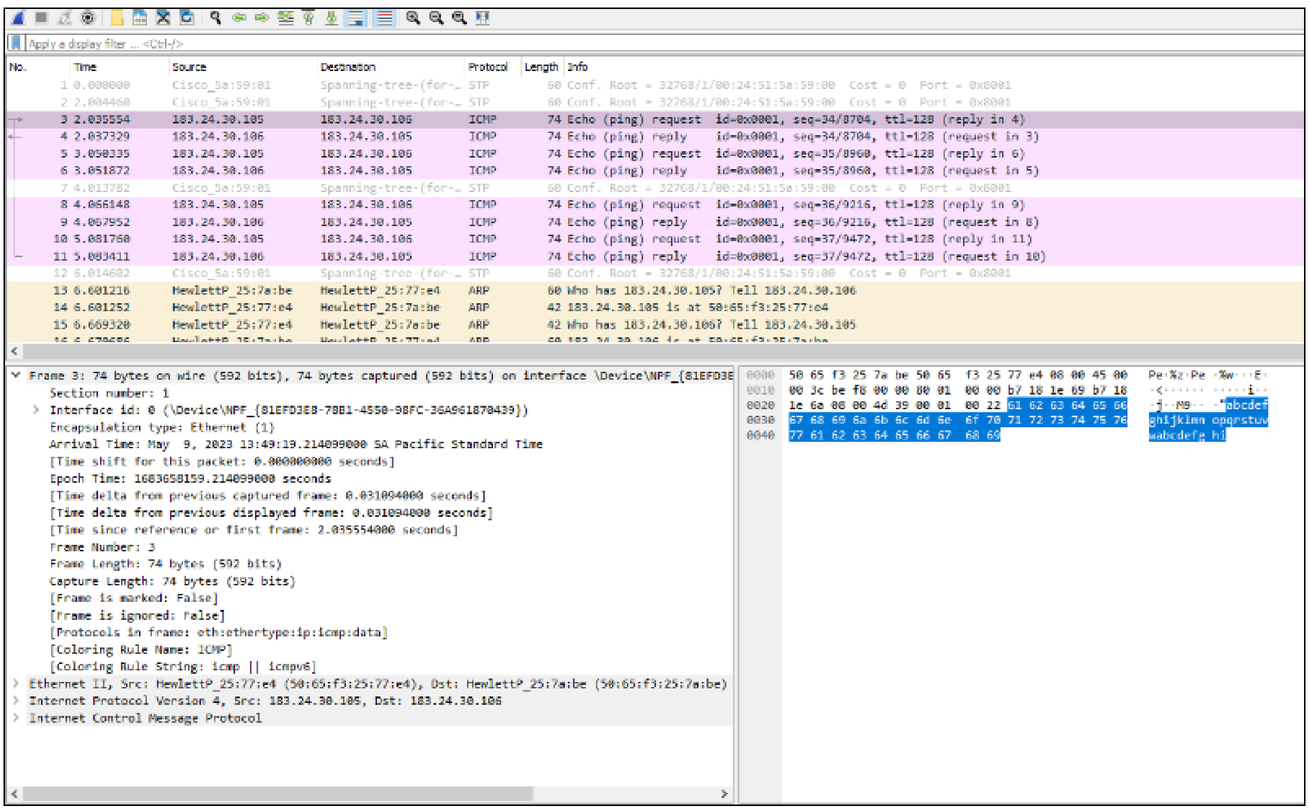
```
nged state to up
Switch#ena
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname Switch1
Switch1(config)#end
Switch1#write
*Mar  1 03:25:46.470: %SYS-5-CONFIG_I: Configured from console by co
Switch1#write memory
Building configuration...
[OK]
Switch1#
```

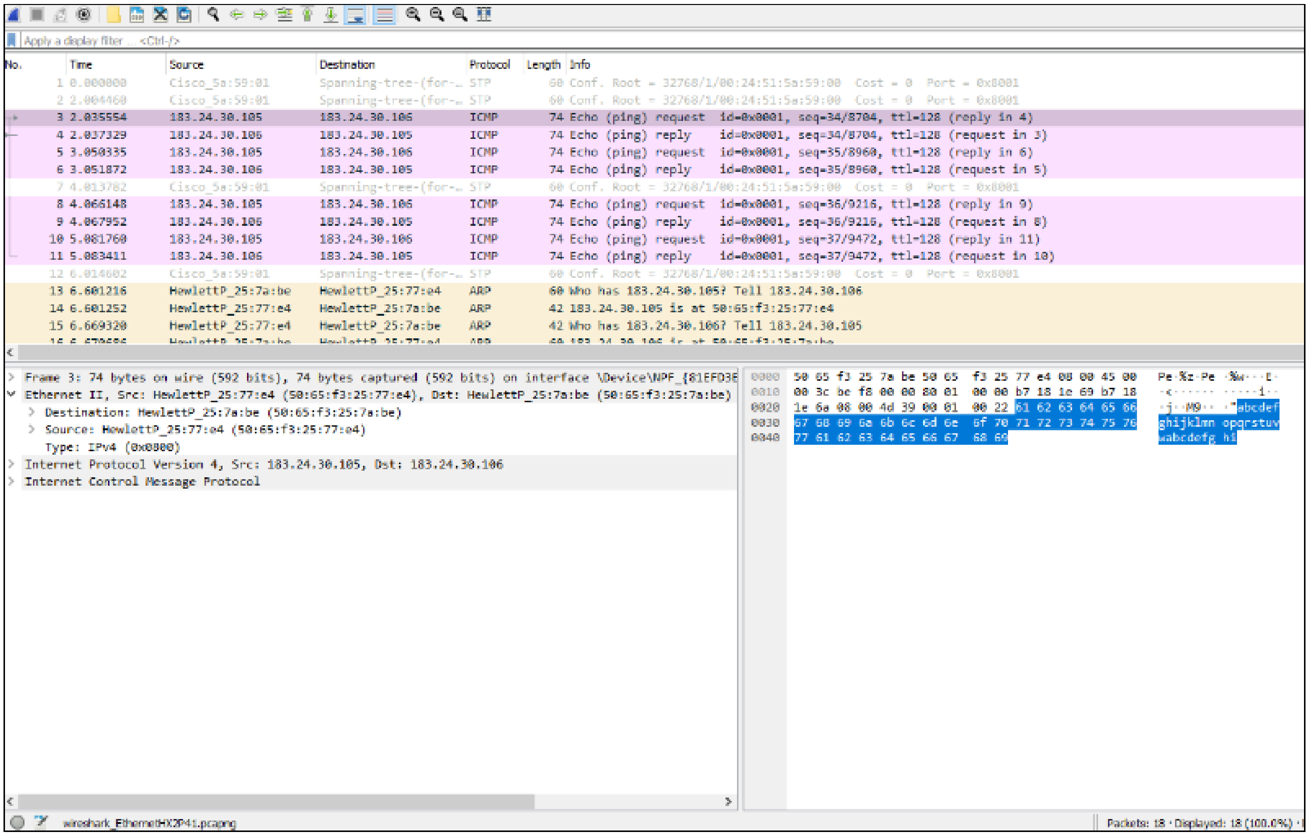
When configuring the system, we worked with a couple of colleagues to connect the switches via the rack and test connections with the `ping` command, thus validating that we have a small network between the four devices, as shown in the diagram.



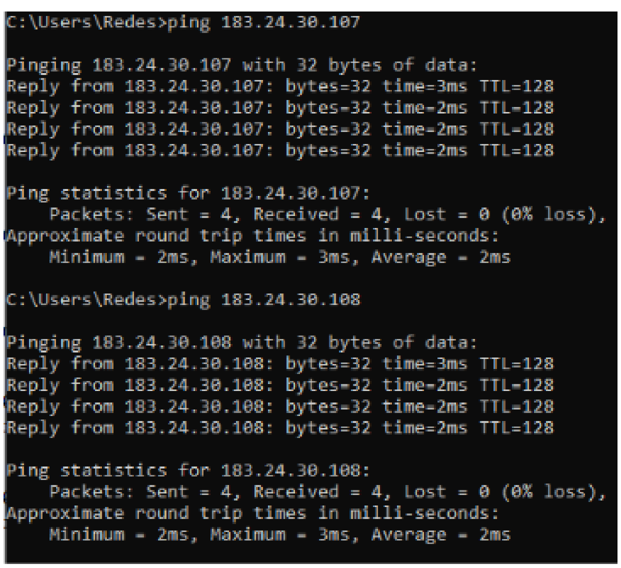


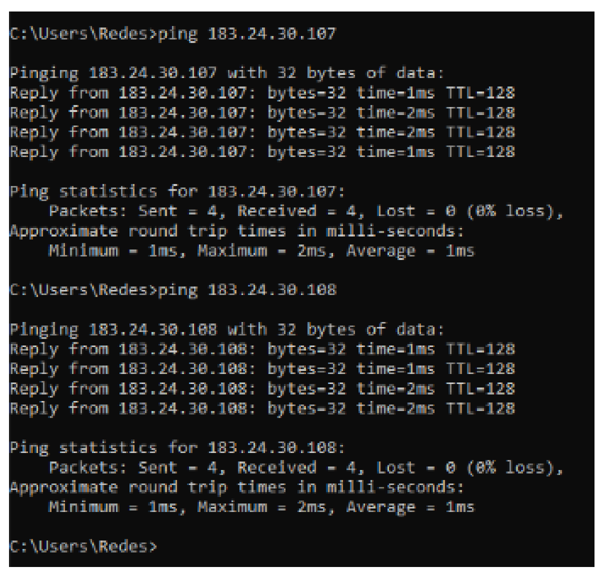
At this point, we have a multi-device system that allows us to use traffic monitoring tools like Wireshark. In this part, we need to view the content of the intercepted frames and obtain MAC addresses, error control, and other information.





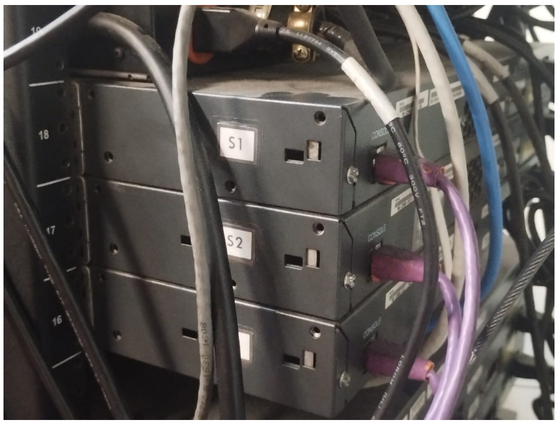
Finally, we just need to validate the connections with the other groups, since we already understand the internal structure of the frames. Therefore, we use ping between the four connected computers and validate it’s functionality.



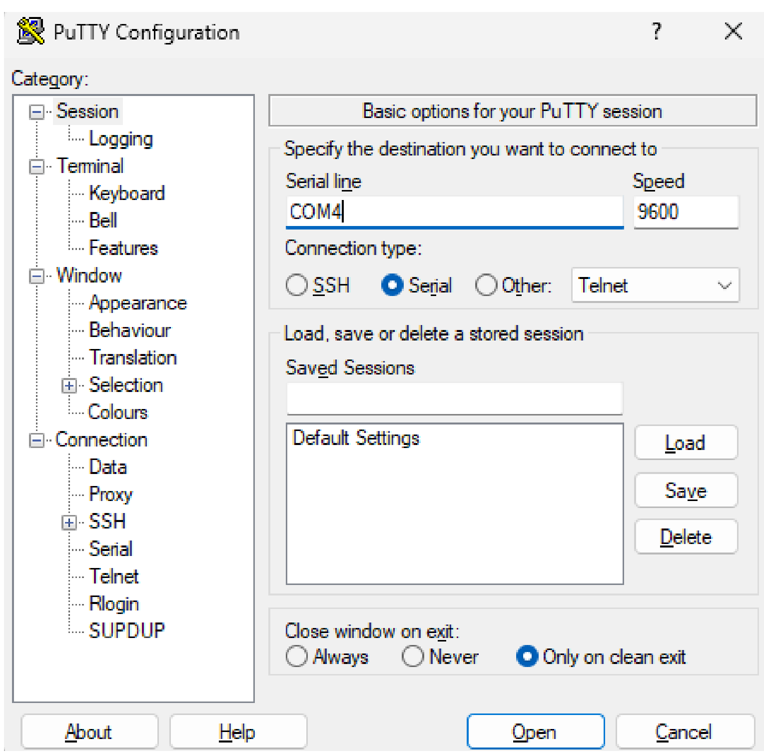


BASIC SWITCH CONFIGURATION II

For this section, and as in the previous lab, we'll configure the routers to begin testing the interconnections based on the lab switches. To do this, we'll repeat the same configuration procedure, starting with logging into PuTTY. Obviously, we need to connect the console cables to our table in the rack.



This way, we can start the application we're connecting to using the SSH protocol. In this case, we simply select the serial connection and the connector name, and it should work.



Now we need to execute the commands required to complete the configurations requested in the lab guide. These allow us to connect to other routers through the already configured switches.

```
Router>
Router>enable
Router#configure terminal Espinosa
                          ^
% Invalid input detected at '^' marker.

Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname Espinosa
Espinosa(config)#enable
% Incomplete command.

Espinosa(config)#clear
                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#show version
                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#enable password cisco
Espinosa(config)#line console 0
Espinosa(config-line)#password claveC
Espinosa(config-line)#login
Espinosa(config-line)#logging synchronouns
Translating "synchronouns"...domain server (255.255.255.255)
                                        ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#line vty 0 4
Espinosa(config-line)#password claveT
Espinosa(config-line)#login
Espinosa(config-line)#logging synchornous
Translating "synchornous"...domain server (255.255.255.255)
                                   ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#no ip domain-lookup
Espinosa(config)#banner motd #Buenas, practica lab 07#
Espinosa(config)#
```

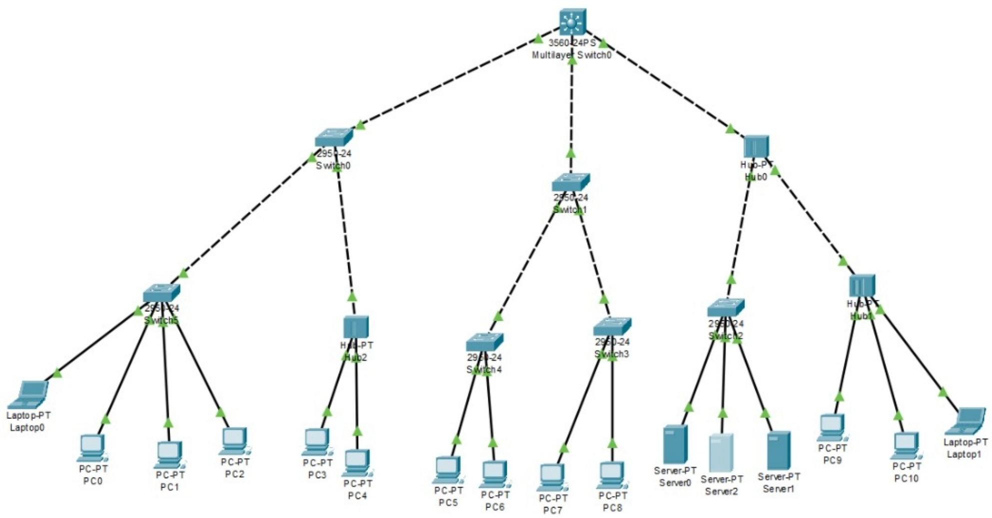
```
Espinosa(config)#show version
                  ^
% Invalid input detected at '^' marker.

Espinosa(config)#enable password cisco
Espinosa(config)#line console 0
Espinosa(config-line)#password claveC
Espinosa(config-line)#login
Espinosa(config-line)#logging synchronouns
Translating "synchronouns"...domain server (255.255.255.255)
                                          ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#line vty 0 4
Espinosa(config-line)#password claveT
Espinosa(config-line)#login
Espinosa(config-line)#logging synchornous
Translating "synchornous"...domain server (255.255.255.255)
                                 ^
% Invalid input detected at '^' marker.

Espinosa(config-line)#logging synchronous
Espinosa(config-line)#exit
Espinosa(config)#no ip domain-lookup
Espinosa(config)#banner motd #Buenas, practica lab 07#
Espinosa(config)#interface GigabithB?
% Unrecognized command
```

```
Espinosa(config)#interface GigabitEthernet0/0
Espinosa(config-if)#description #Equipo de 600 hosts
Espinosa(config-if)#ip address 88.0.4.0 255.255.252.0
Bad mask /22 for address 88.0.4.0
Espinosa(config-if)#ip address 88.0.4.1 255.255.252.0
Espinosa(config-if)#interface GigabitEthernet0/1
Espinosa(config-if)#description #Equipo de 4000 hosts
Espinosa(config-if)#ip address 88.0.16.1 255.255.240.0
Espinosa(config-if)#no shutdown
Espinosa(config-if)#exit
Espinosa(config)#description #Equipo de 4000 hosts
*Apr  7 22:26:53.911: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to down
Espinosa(config)#
*Apr  7 22:27:02.151: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to up
*Apr  7 22:27:03.151: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
Espinosa(config)#interface GigabithEthernet0/0
                                   ^
% Invalid input detected at '^' marker.

Espinosa(config)#interface GigabitEthernet0/0
Espinosa(config-if)#description #Equipo de 600 hosts
Espinosa(config-if)#ip address 88.0.4.1 255.255.252.0
Espinosa(config-if)#no shutdown
Espinosa(config-if)#exit
Espinosa(config)#
*Apr  7 22:27:33.487: %LINK-3-UPDOWN: Interface GigabitEthernet0/0, changed state to down
Espinosa(config)#
*Apr  7 22:27:42.151: %LINK-3-UPDOWN: Interface GigabitEthernet0/0, changed state to up
*Apr  7 22:27:43.151: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
Espinosa(config)#
```

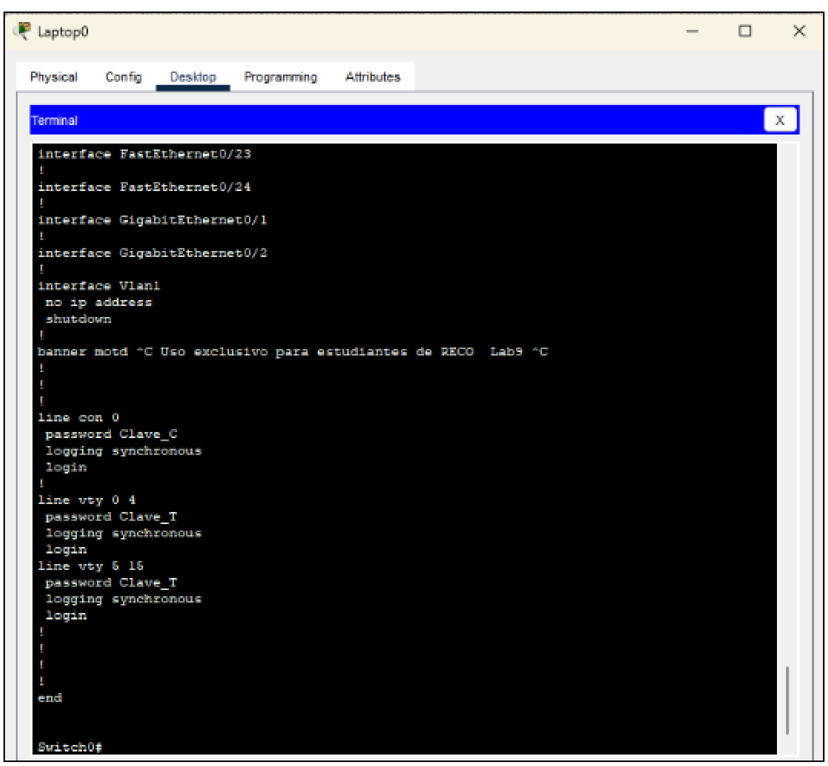
This section doesn't explain much because it's something we'd already covered in the previous lab, as mentioned above. Even so, we decided to include a quick explanation for general understanding of the router and swiches configuration.

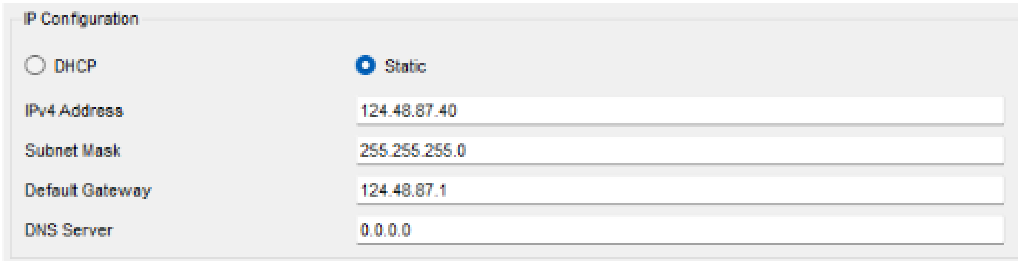
LARGER SWITCH NETWORKS

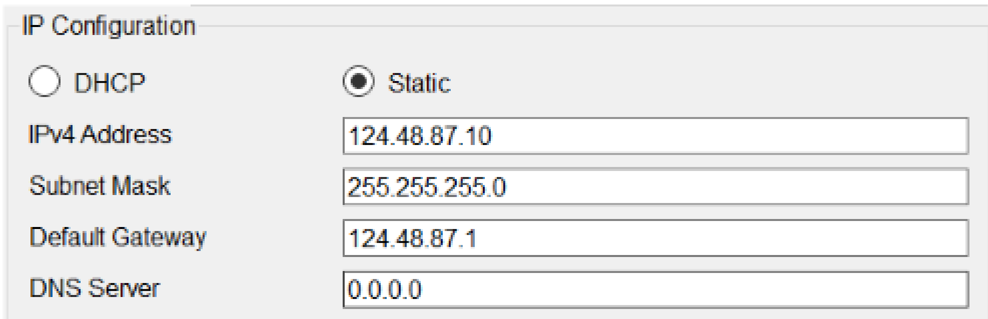
In this first section of Packet Tracer, we'll understand how switches and some related algorithms work. To do this, we need to replicate the diagram shown in the guide.



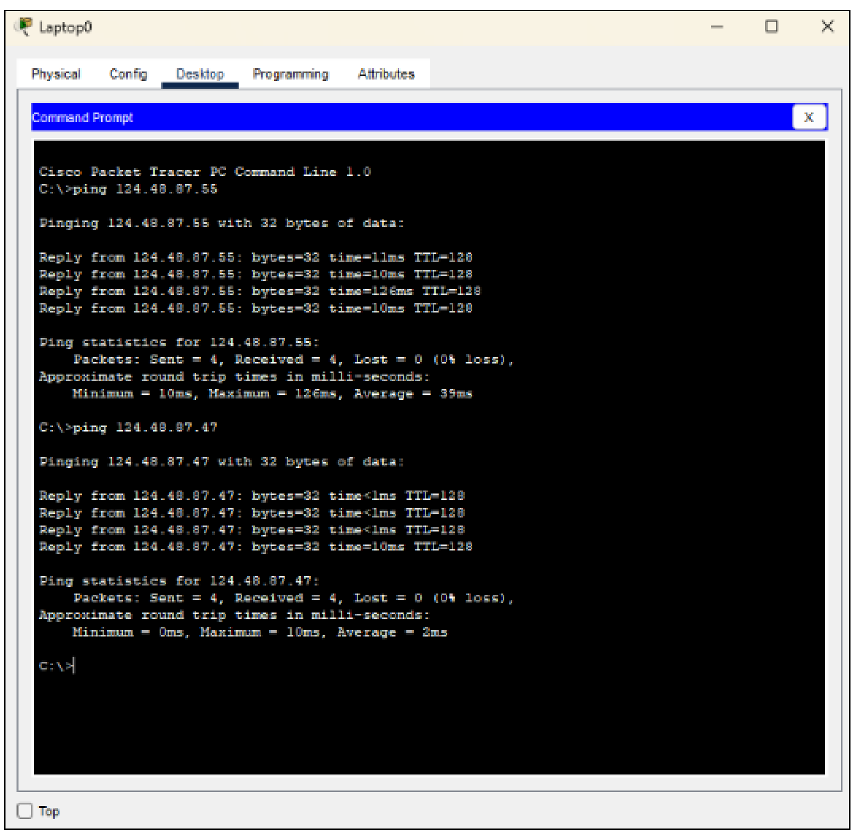
Now, we need to perform the configurations we saw in previous labs on all the switches. Additionally, we configure the devices based on the ranges and parameters in the guide, so that each device accesses the same network.





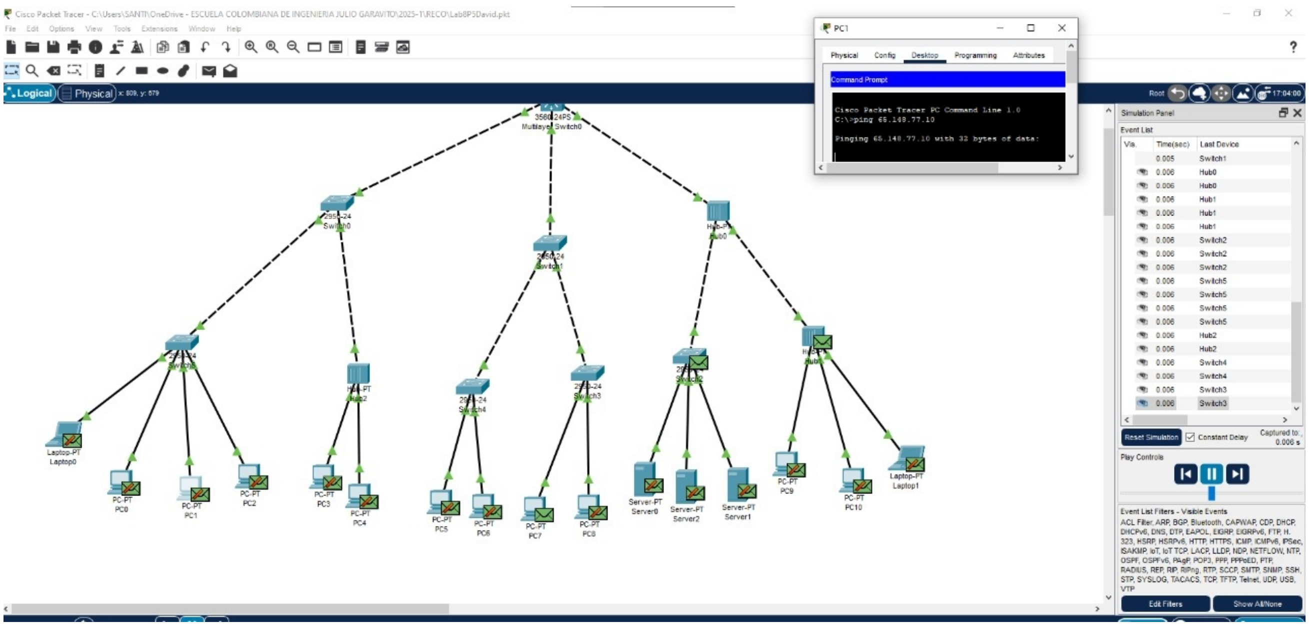


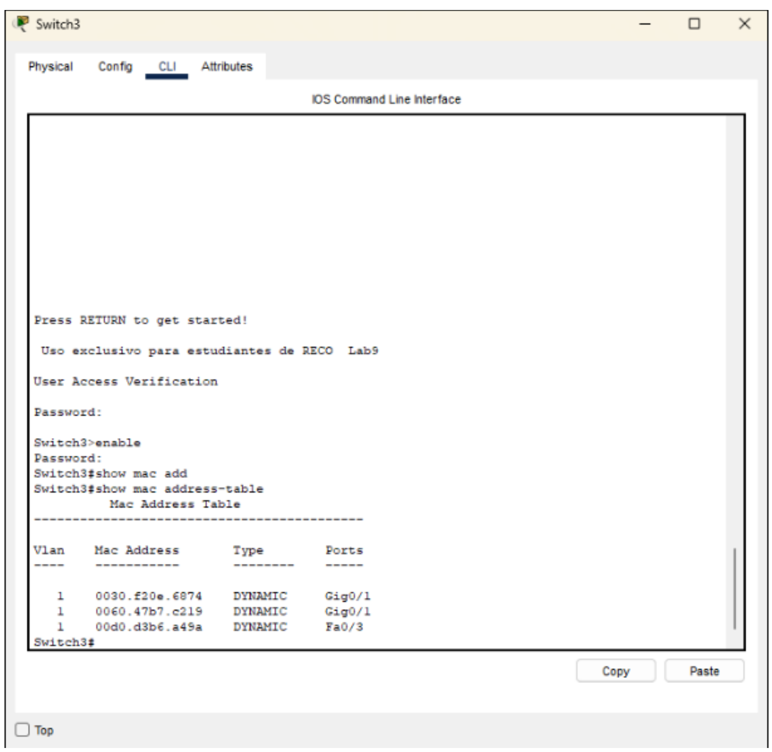
We validate the equipment connection by pinging between the first and seventh equipment in our Packet Tracer network.



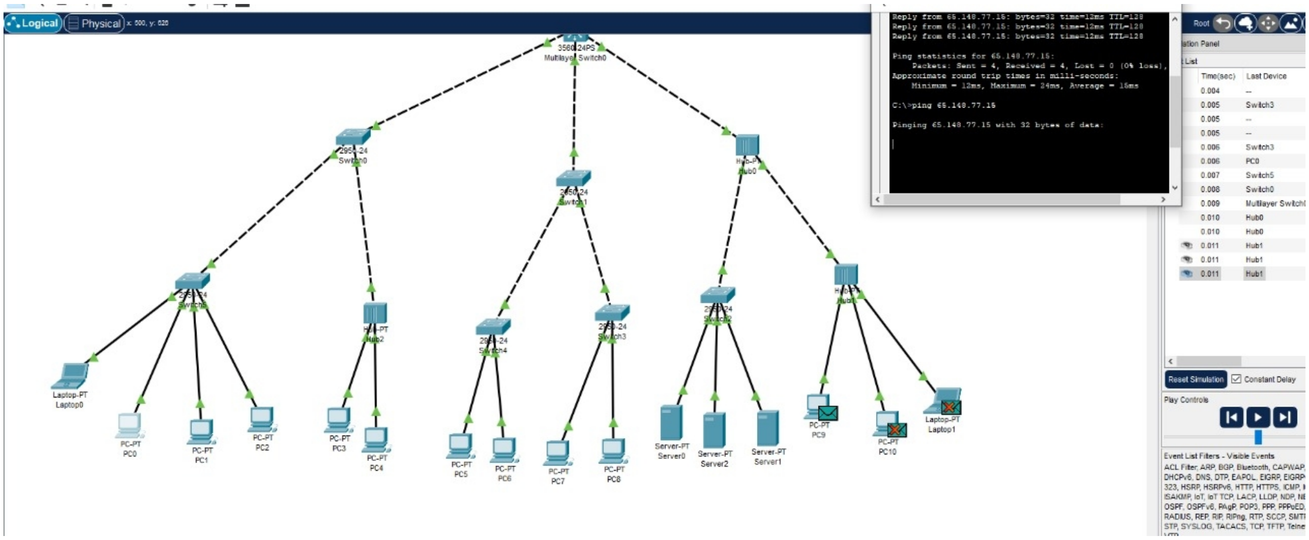
At this point, using simulation mode, we review the network behavior and the format of an Ethernet frame when sending frames. We identify the behavior of the switches and the hash tables for each of the requested cases:

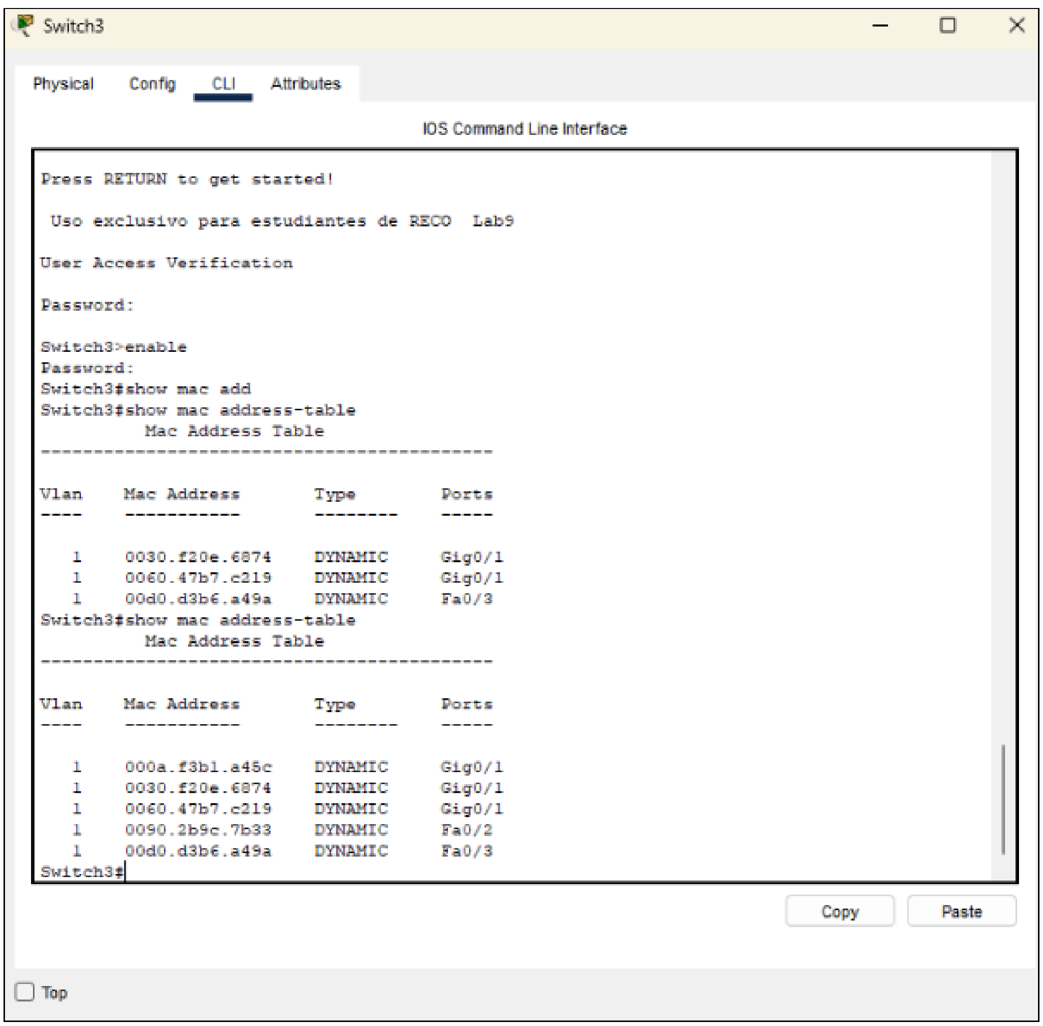
- From PC1 to PC7:



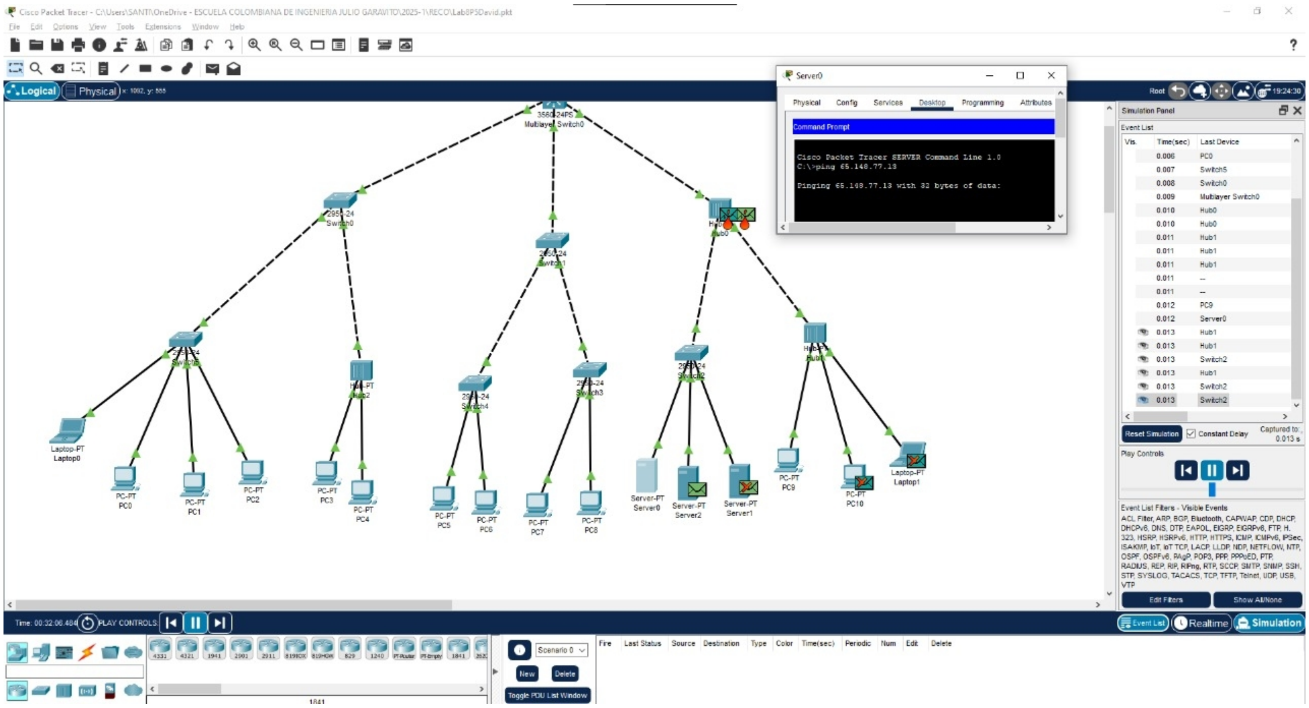


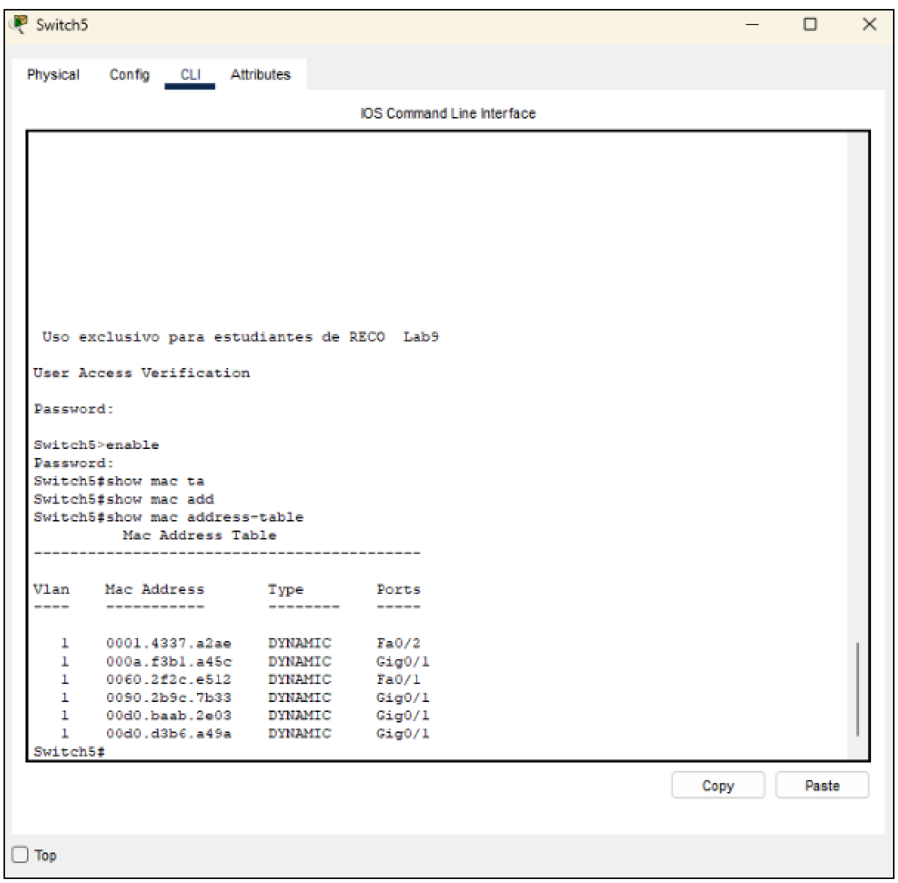
- From PC0 to PC9:



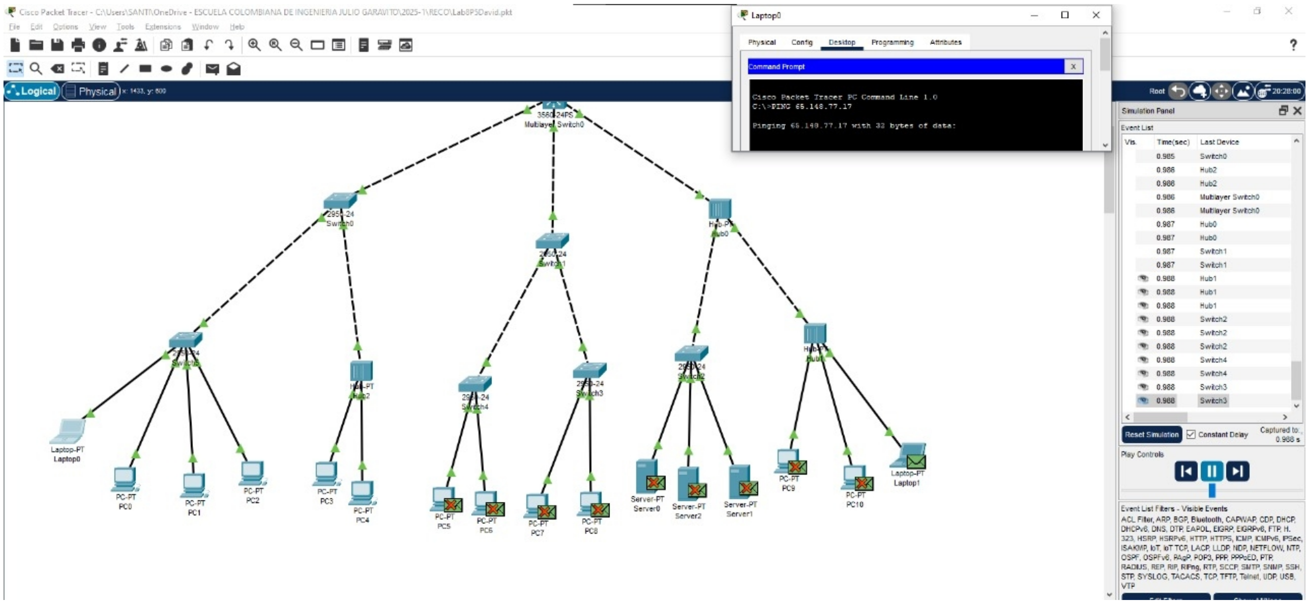


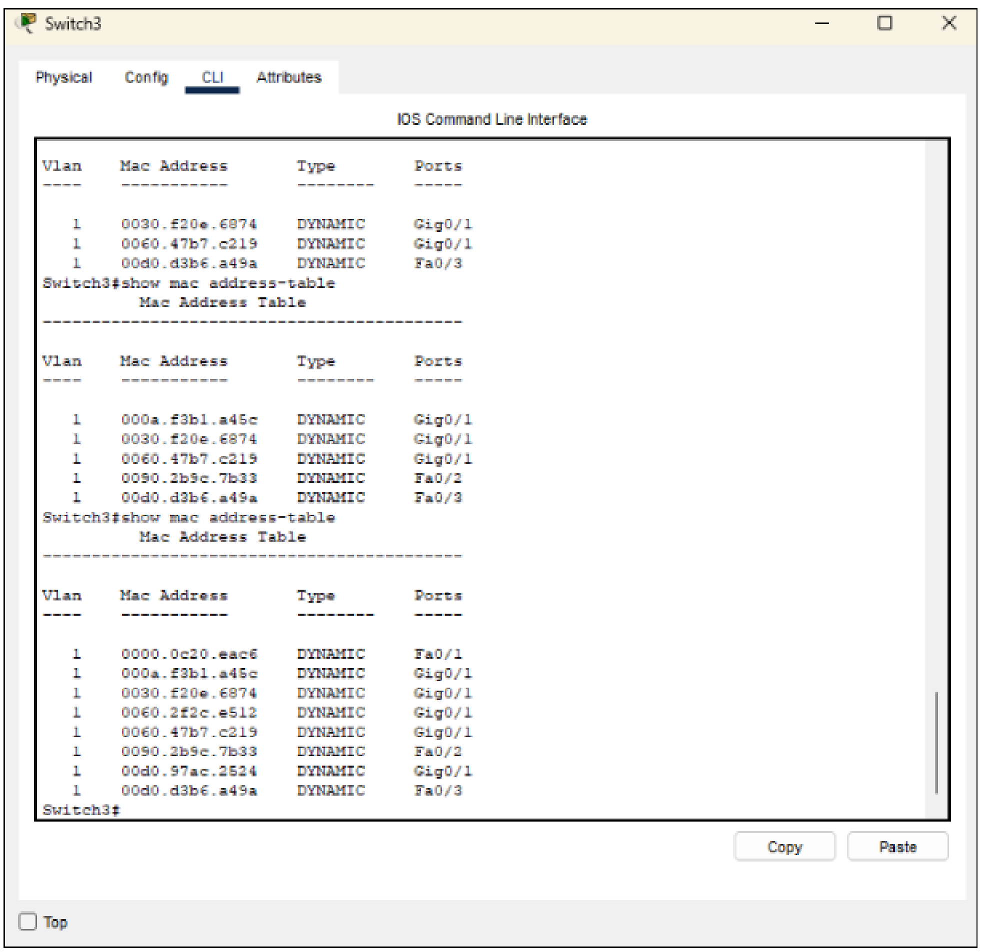
- From Server0 to Server1:





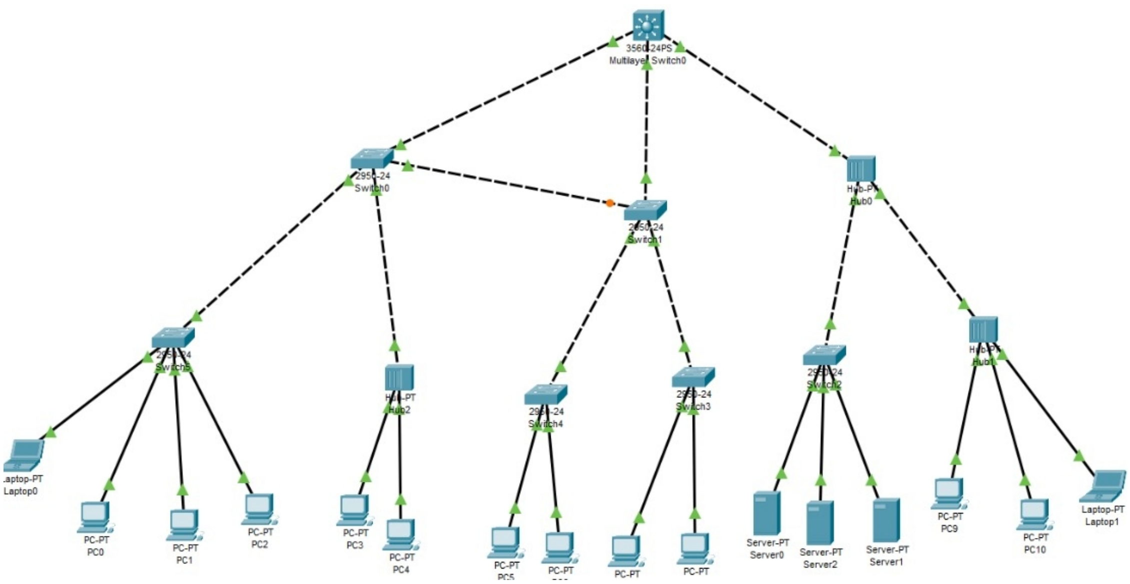
- From Laptop0 to Laptop1:



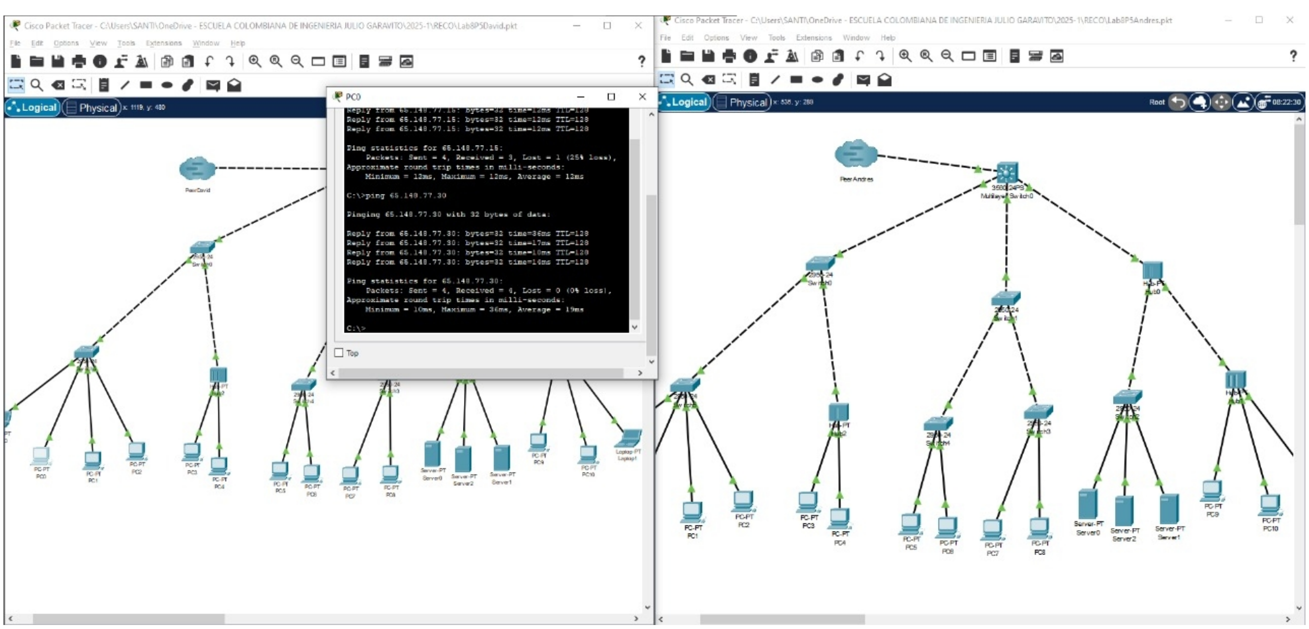



```
Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0030.f20e.6874    DYNAMIC     Gig0/1
   1    0060.47b7.c219    DYNAMIC     Gig0/1
   1    00d0.d3b6.a49a    DYNAMIC     Fa0/3
Switch3#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    000a.f3b1.a45c    DYNAMIC     Gig0/1
   1    0030.f20e.6874    DYNAMIC     Gig0/1
   1    0060.47b7.c219    DYNAMIC     Gig0/1
   1    0090.2b9c.7b33    DYNAMIC     Fa0/2
   1    00d0.d3b6.a49a    DYNAMIC     Fa0/3
Switch3#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0000.0c20.eac6    DYNAMIC     Fa0/1
   1    000a.f3b1.a45c    DYNAMIC     Gig0/1
   1    0030.f20e.6874    DYNAMIC     Gig0/1
   1    0060.2f2c.e512    DYNAMIC     Gig0/1
   1    0060.47b7.c219    DYNAMIC     Gig0/1
   1    0090.2b9c.7b33    DYNAMIC     Fa0/2
   1    00d0.97ac.2524    DYNAMIC     Gig0/1
   1    00d0.d3b6.a49a    DYNAMIC     Fa0/3
Switch3#
```

To wrap things up, let's look at the operation of the spanning tree algorithm, as we interconnected the switches to see what happened in the simulation. In this simulation, we saw that access from switch 0 to switch 1 was blocked, as the algorithm detects two possible paths and determines a single mode of operation, thus avoiding collisions or cycles in the networks.



Finally, we used the cloud function to connect the two Packet Tracer modules requested. These two modules allow us to ping back and forth following the same tree pattern seen in this section, so we were able to create a functional interconnected network.



VLAND CONFIGURATION

To this section and considering the reference diagram, we have to create VLANs so we can connect and communicate with different devices within the room. In this case, we have two VLANs with IDs 50 and 55, respectively.

Initially, we need to configure our router, as access to this network is done manually, using the `***access vland***` command on each network interface representing a device.

```
Switch1(config-if)#switchport access vlan 10
% Access VLAN does not exist. Creating vlan 10
Switch1(config-if)#exit
Switch1(config)#interface fastEthernet0/2
Switch1(config-if)#switchport access vlan 20
% Access VLAN does not exist. Creating vlan 20
Switch1(config-if)#exit
Switch1(config)#end
Switch1#
*Mar  1 04:41:13.634: %SYS-5-CONFIG_I: Configured from console by console
Switch1#write memory
Building configuration...
[OK]
Switch1#
```

We can view network information using the `show vland brief` command, where we validate which ones are active and which ports are assigned to them.

```
VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/1, Fa0/3, Fa0/4, Fa0/5
                                                Fa0/6, Fa0/7, Fa0/9, Fa0/10
                                                Fa0/11, Fa0/12, Fa0/13, Fa0/14
                                                Fa0/15, Fa0/16, Fa0/17, Fa0/18
                                                Fa0/19, Fa0/20, Fa0/21, Fa0/22
                                                Fa0/23, Fa0/24, Gi0/1, Gi0/2
10   VLAN0010                         active    Fa0/8
20   VLAN0020                         active    Fa0/2
50   sistemas                         active
55   otros                            active
1002 fddi-default                     act/unsup
1003 token-ring-default               act/unsup
1004 fddinet-default                  act/unsup
1005 trnet-default                    act/unsup
Switch1#
```

Now we configure the port to be a trunk, which is a point-to-point link between two network devices that carries more than one VLAN. This basically allows multiple ports to connect to a single point without overload the net.

```
interface GigabitEthernet0/1
 description "Conexion a switch 1"
 switchport mode trunk
!
interface GigabitEthernet0/2
 description "Conexion a switch 2"
 switchport mode trunk
```

Finally, we can perform a connectivity test by pinging the console to a machine that is on the same VLAN as each device, keeping in mind that the entire class has its respective devices on the corresponding network.

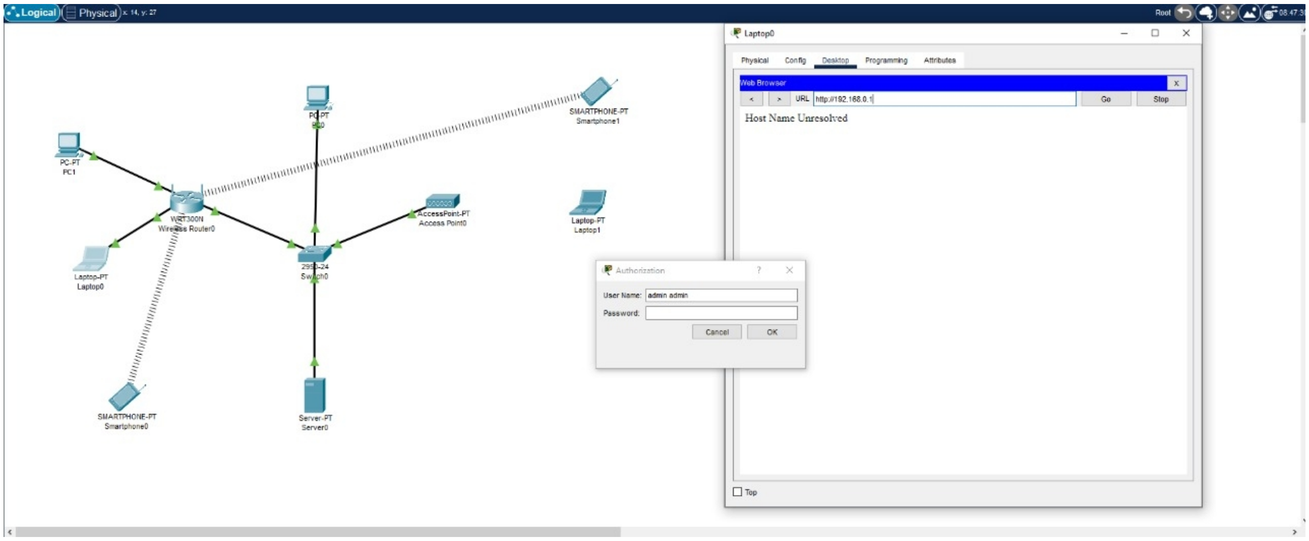
```
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
Control-C
^C
C:\Users\Redes>ping 10.2.77.16

Pinging 10.2.77.16 with 32 bytes of data:
Reply from 10.2.77.16: bytes=32 time<1ms TTL=128
Reply from 10.2.77.16: bytes=32 time=1ms TTL=128

Ping statistics for 10.2.77.16:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
Control-C
^C
C:\Users\Redes>ping 10.2.77.17

Pinging 10.2.77.17 with 32 bytes of data:
Reply from 10.2.77.17: bytes=32 time=2ms TTL=128
Reply from 10.2.77.17: bytes=32 time=1ms TTL=128

Ping statistics for 10.2.77.17:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 2ms, Average = 1ms
```

```
Pinging 10.2.77.12 with 32 bytes of data:
Reply from 10.2.77.12: bytes=32 time=1ms TTL=128
Reply from 10.2.77.12: bytes=32 time=1ms TTL=128

Ping statistics for 10.2.77.12:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

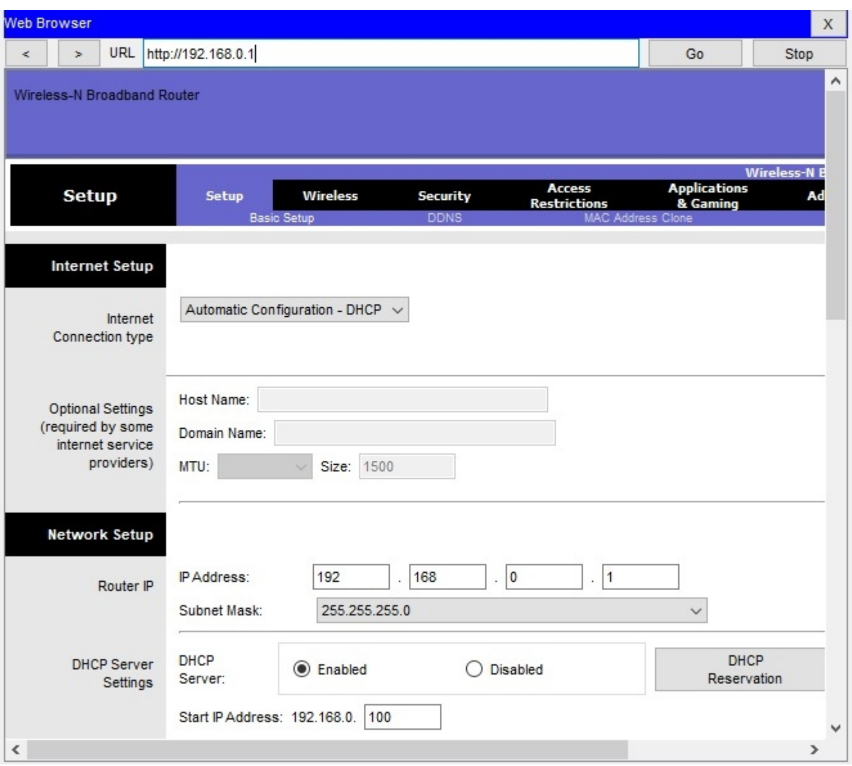
```
C:\Users\Redes>ping 10.2.77.16

Pinging 10.2.77.16 with 32 bytes of data:
Reply from 10.2.77.16: bytes=32 time=1ms TTL=128
Reply from 10.2.77.16: bytes=32 time=1ms TTL=128
Reply from 10.2.77.16: bytes=32 time=1ms TTL=128
Reply from 10.2.77.16: bytes=32 time=1ms TTL=128

Ping statistics for 10.2.77.16:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

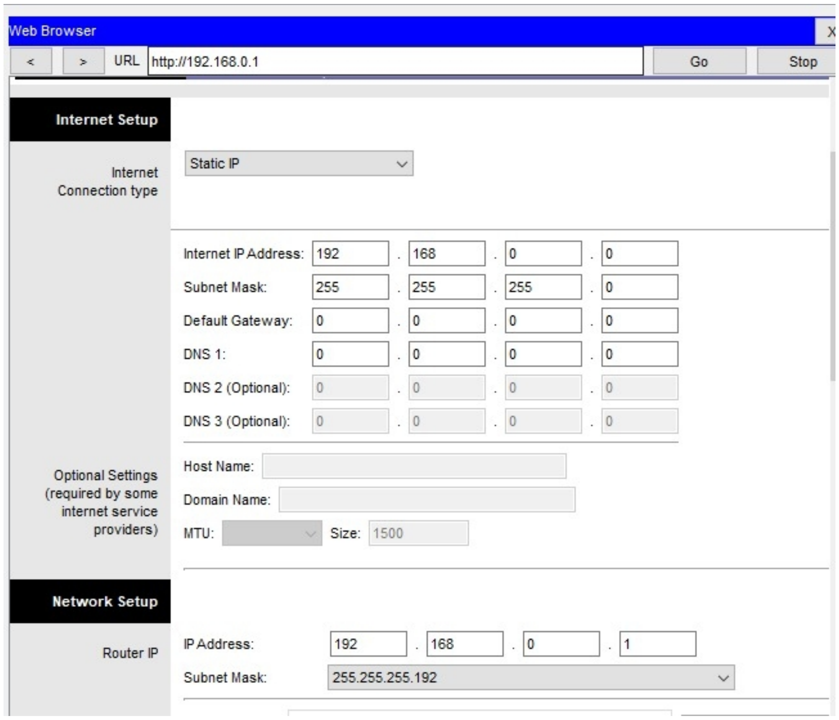
BASIC WIFI CONFIGURATION

In this section, we'll use Packet Tracer to represent and analyze wireless networks using the structure suggested in the guide. To do it, we need to follow the parameter indications for each of the elements in the diagram with a laptop device.

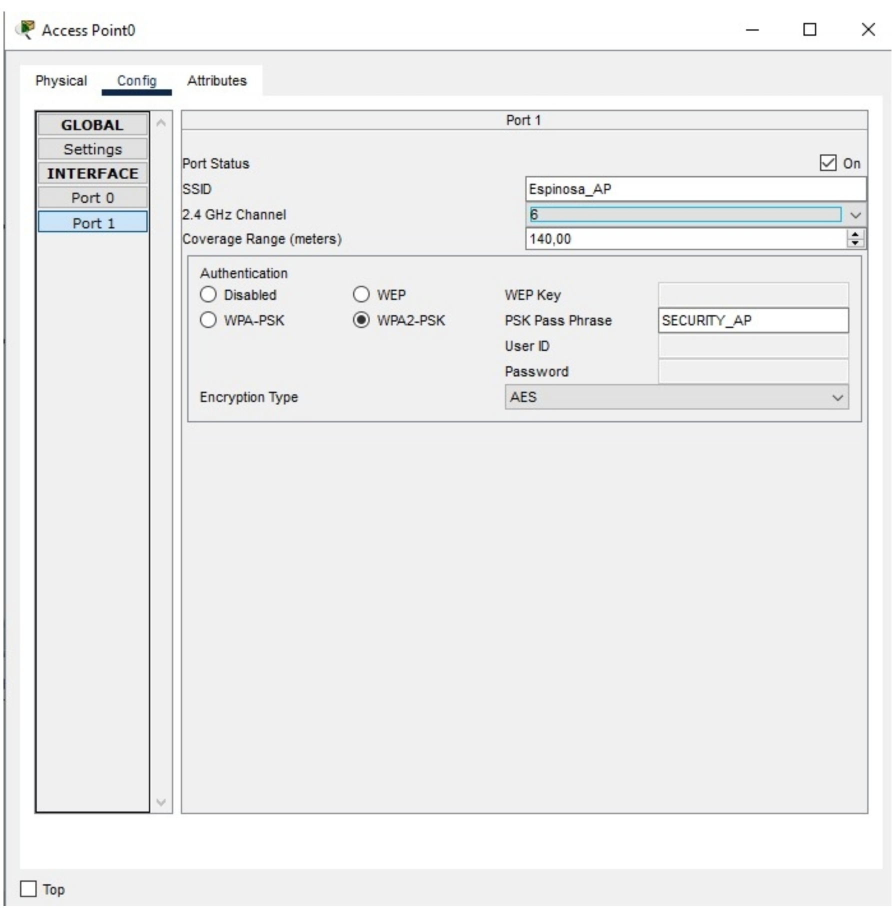


Using the laptop, we'll configure the requested settings by connecting to the router via the terminal. We'll configure the wired network to ensure a secure connection and also enter the subnet information to ensure proper operation. On the other hand, configure the wireless network with the same parameters so that our entire system is under the same configuration.

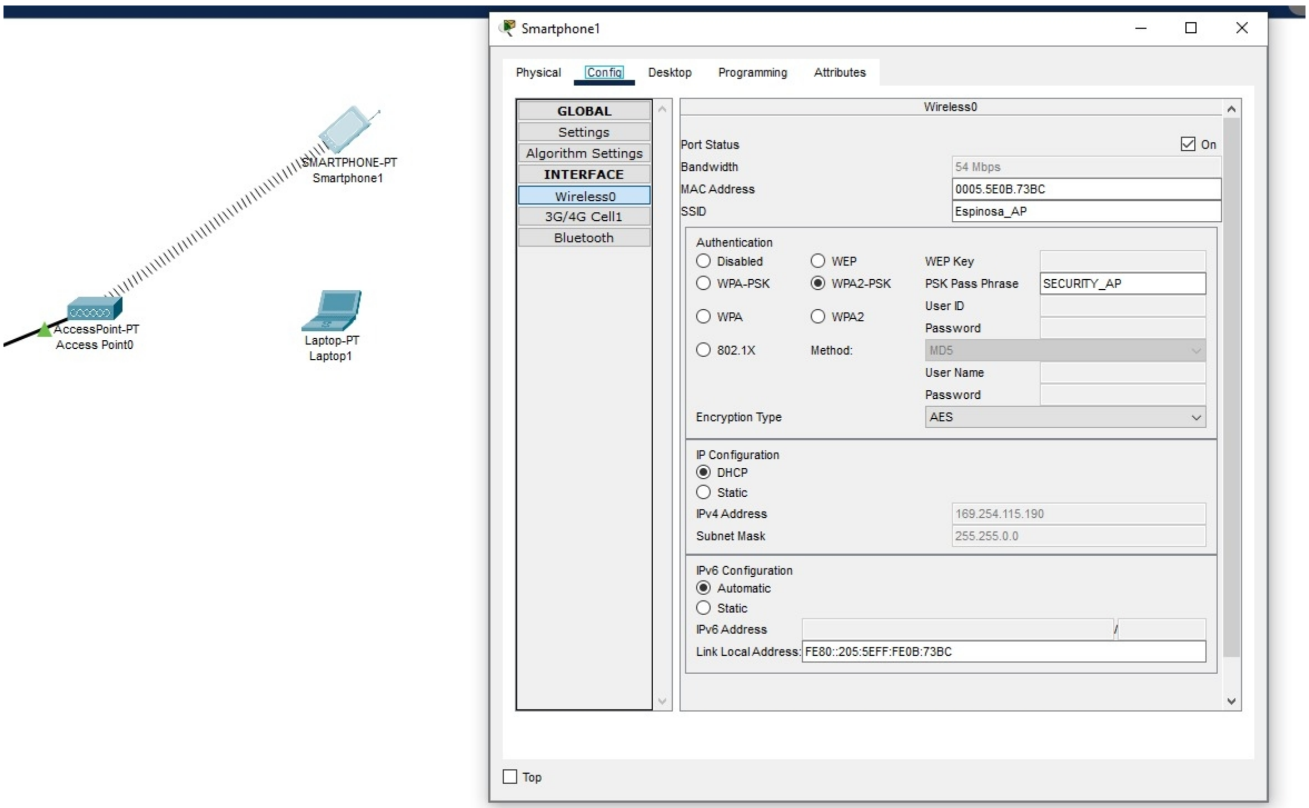




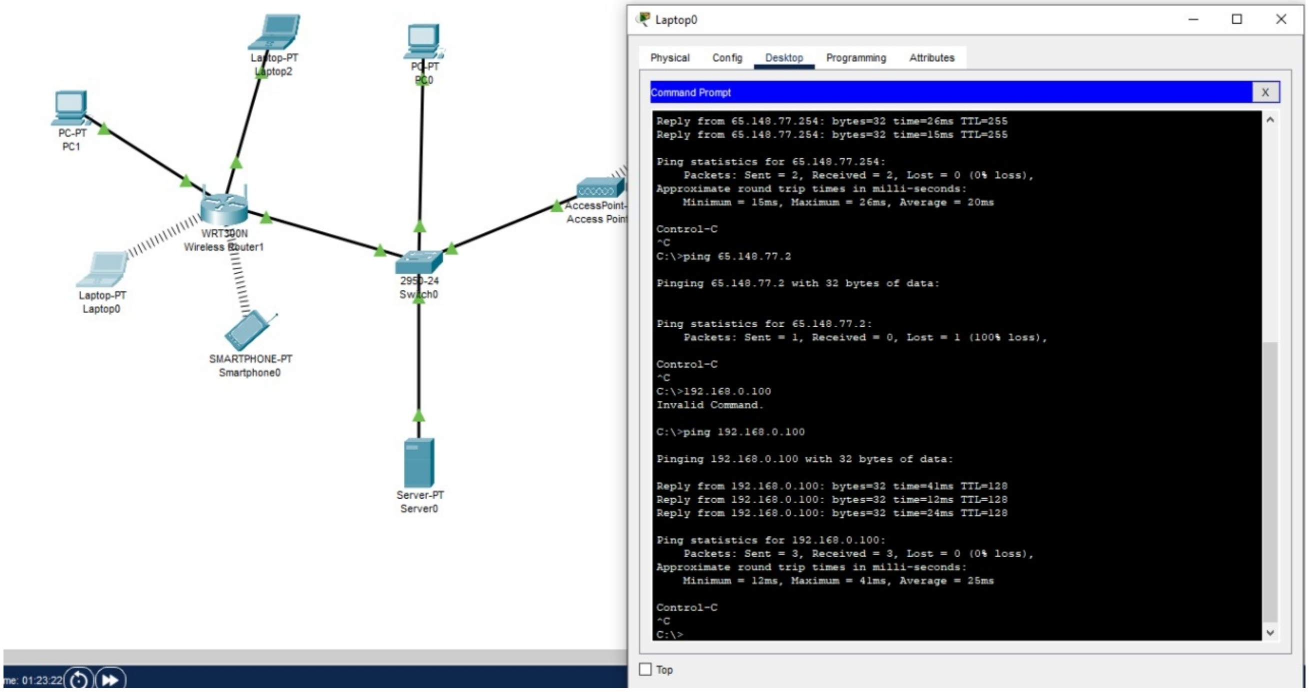
Now we configure the wireless network repeater, this way we can validate connections with devices such as the SSID, network and router IP, range, WPA2-PSK authentication with AES, passwords with SECURITY_R and other settings.



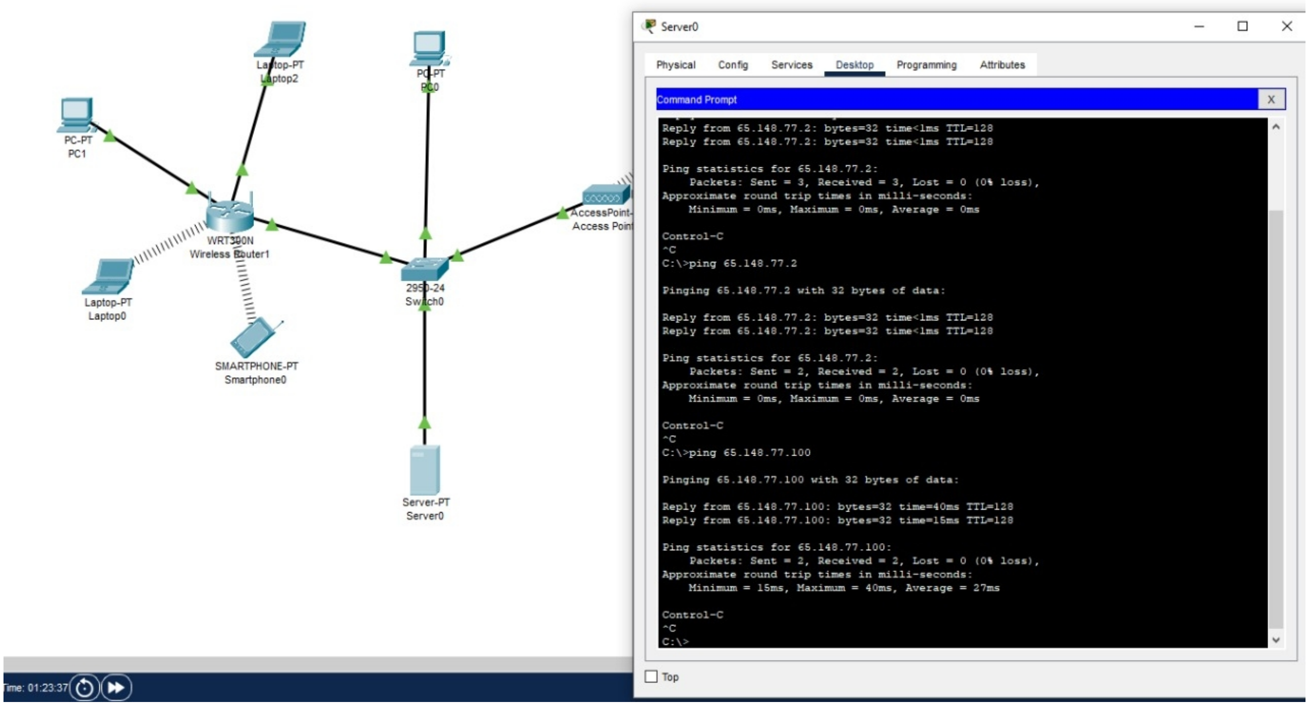
To finish the setup, we need to connect the wireless devices to the network we previously configured. We set the same parameters as for the repeater, but focusing on the network connection.



Finally, we ping between devices on the two present networks (wired and wireless) to validate the correct operation of the Packet Tracer setup,



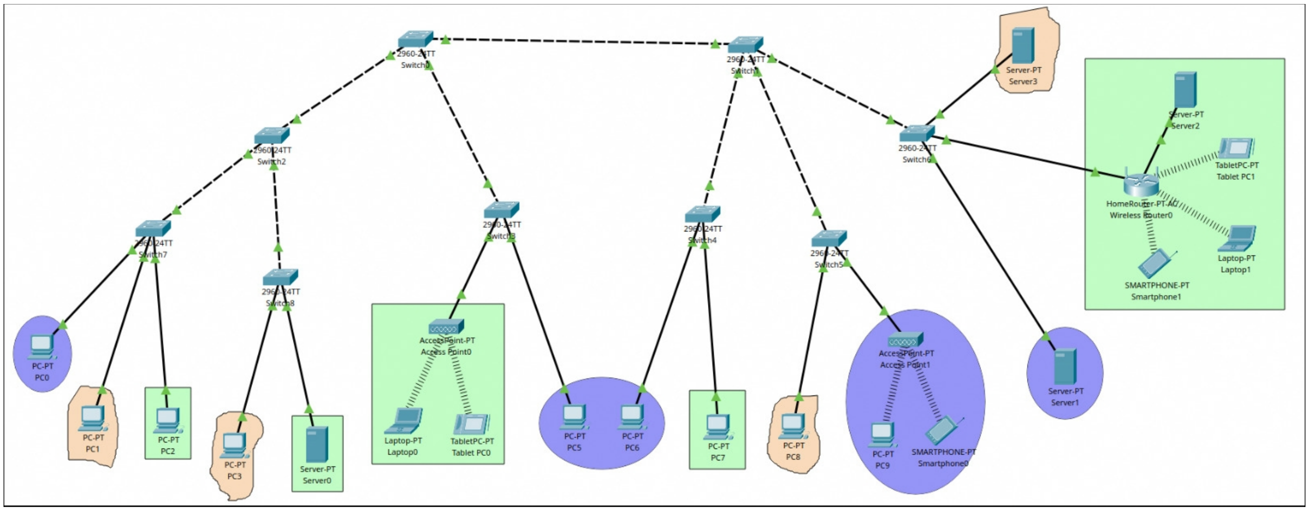
Laptop-PT
Laptop2
PC-PT
PC0
PC-PT
PC1
AccessPoint-
Access Point
WRT300N
Wireless Router1
2950-24
Switch0
Laptop-PT
Laptop0
SMARTPHONE-PT
Smartphone0
Server-PT
Server0
me: 01:23:22
Laptop0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
Reply from 65.148.77.254: bytes=32 time=26ms TTL=255
Reply from 65.148.77.254: bytes=32 time=15ms TTL=255
Ping statistics for 65.148.77.254:
Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 15ms, Maximum = 26ms, Average = 20ms
Control-C
^C
C:\>ping 65.148.77.2
Pinging 65.148.77.2 with 32 bytes of data:
Ping statistics for 65.148.77.2:
Packets: Sent = 1, Received = 0, Lost = 1 (100% loss),
Control-C
^C
C:\>192.168.0.100
Invalid Command.
C:\>ping 192.168.0.100
Pinging 192.168.0.100 with 32 bytes of data:
Reply from 192.168.0.100: bytes=32 time=41ms TTL=128
Reply from 192.168.0.100: bytes=32 time=12ms TTL=128
Reply from 192.168.0.100: bytes=32 time=24ms TTL=128
Ping statistics for 192.168.0.100:
Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 12ms, Maximum = 41ms, Average = 25ms
Control-C
^C
C:\>
Top



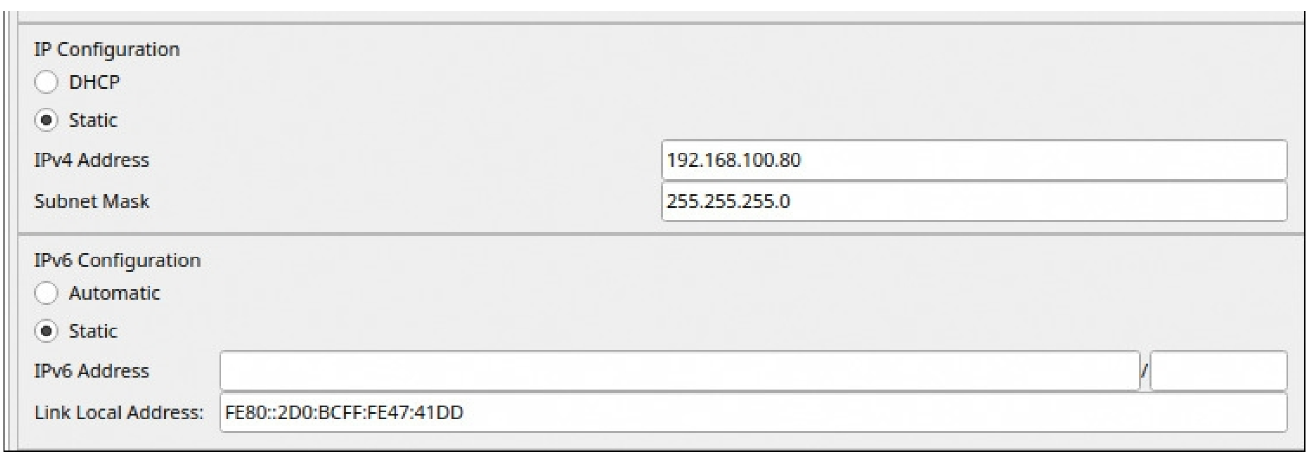
Laptop-PT
Laptop2
PC-PT
PC0
PC-PT
PC1
AccessPoint-
Access Point
WRT300N
Wireless Router1
2950-24
Switch0
Laptop-PT
Laptop0
SMARTPHONE-PT
Smartphone0
Server-PT
Server0
Time: 01:23:37
Server0
Physical
Config
Services
Desktop
Programming
Attributes
Command Prompt
Reply from 65.148.77.2: bytes=32 time<1ms TTL=128
Reply from 65.148.77.2: bytes=32 time<1ms TTL=128
Ping statistics for 65.148.77.2:
Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
Control-C
^C
C:\>ping 65.148.77.2
Pinging 65.148.77.2 with 32 bytes of data:
Reply from 65.148.77.2: bytes=32 time<1ms TTL=128
Reply from 65.148.77.2: bytes=32 time<1ms TTL=128
Ping statistics for 65.148.77.2:
Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
Control-C
^C
C:\>ping 65.148.77.100
Pinging 65.148.77.100 with 32 bytes of data:
Reply from 65.148.77.100: bytes=32 time=40ms TTL=128
Reply from 65.148.77.100: bytes=32 time=15ms TTL=128
Ping statistics for 65.148.77.100:
Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 15ms, Maximum = 40ms, Average = 27ms
Control-C
^C
C:\>
Top

CONFIGURATION OF WIRED & WIRELESS LAN

Having already established two different types of connections, we'll now perform a mixed setup to analyze how a network behaves when it contains both wireless and wired devices. Initially, we'll perform the setup as described in the guide.

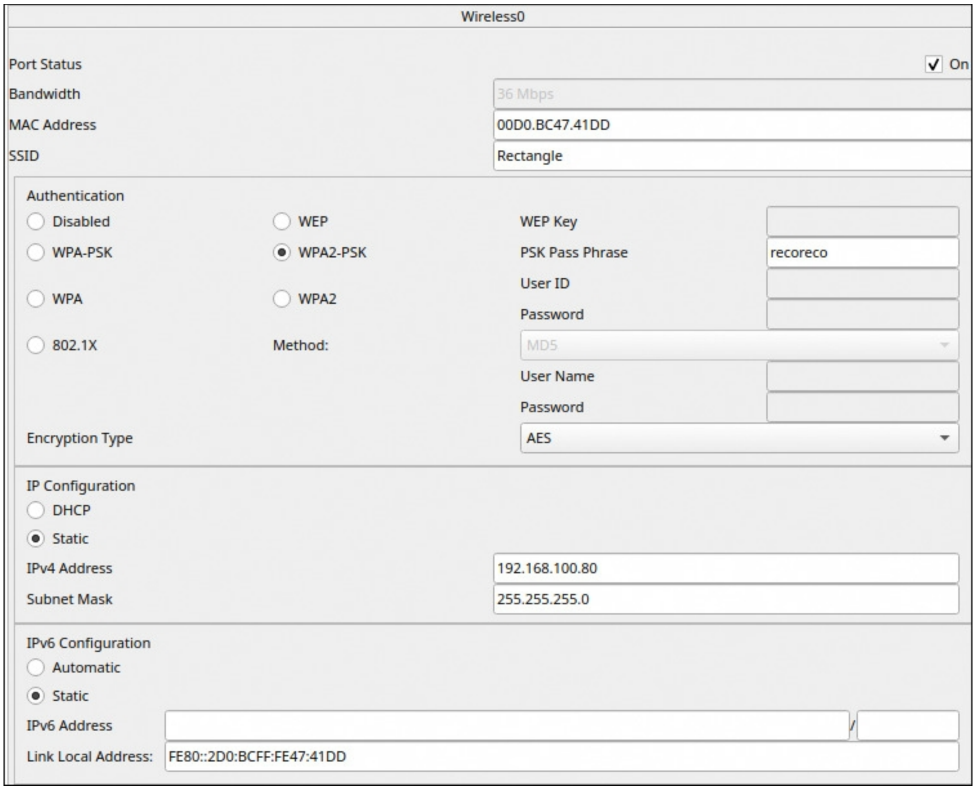


In this system, we have to configure all wired devices with the IP addresses assigned for this case, placing the configurations on each of the devices (diferent image but works as reference).

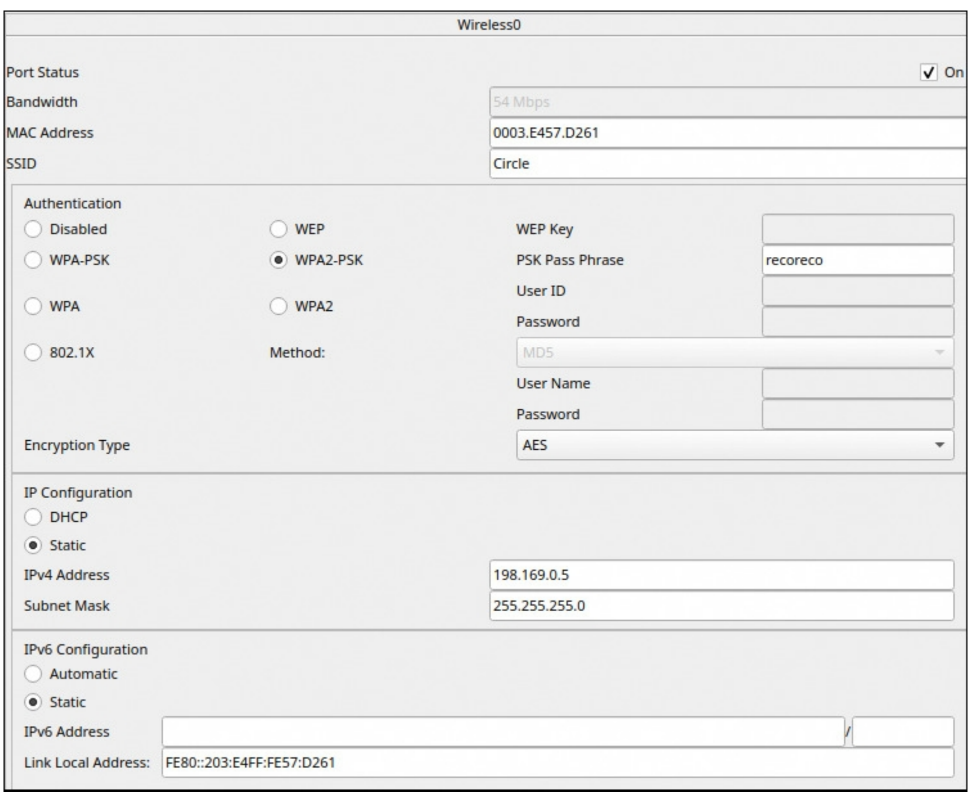


Now that the wired devices have been configured, we need to configure the different wireless elements depending on the color or area designated by the diagram.

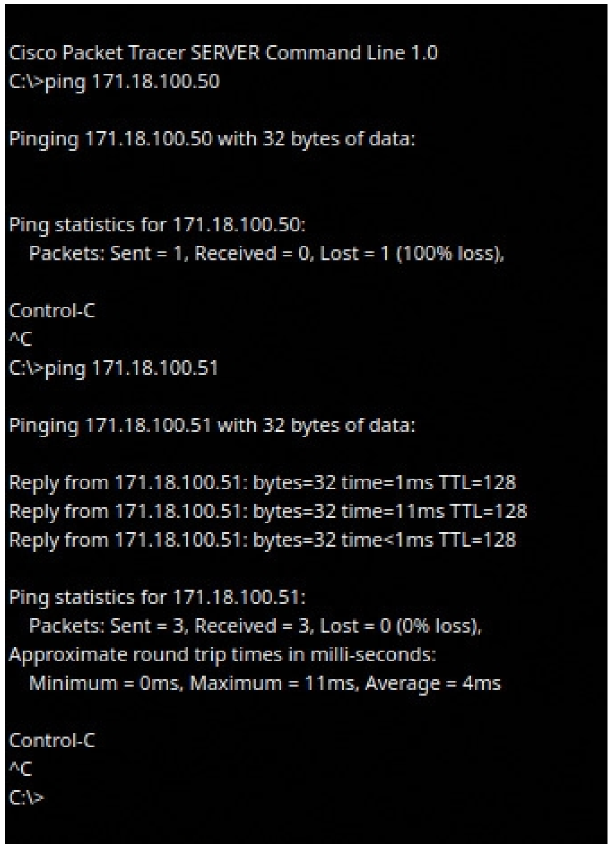
- Green areas:



- Purple areas:

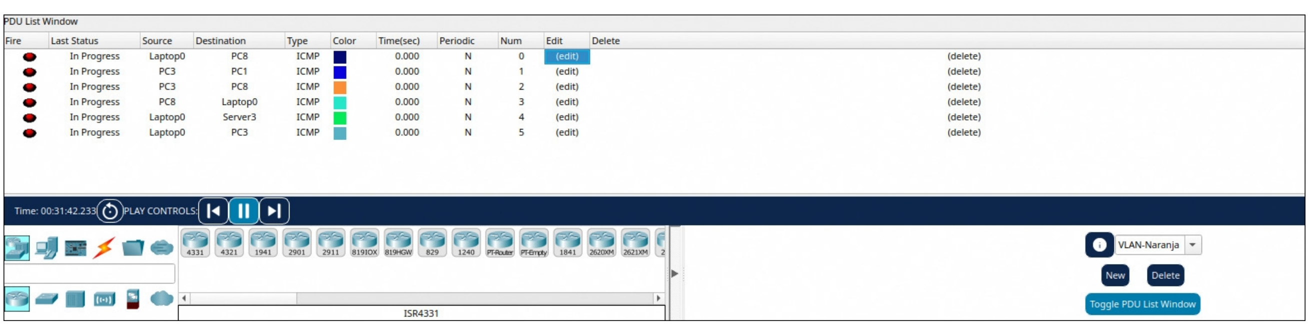


This setup itself only allows you to connect devices that have a common switch, as seen in the image. However, for more remote or wireless networks, it doesn't allow connections without additional configuration, although it follows the same pattern of connecting common switches.


```
Cisco Packet Tracer SERVER Command Line 1.0
C:\>ping 171.18.100.50

Pinging 171.18.100.50 with 32 bytes of data:


Ping statistics for 171.18.100.50:
    Packets: Sent = 1, Received = 0, Lost = 1 (100% loss),

Control-C
^C
C:\>ping 171.18.100.51

Pinging 171.18.100.51 with 32 bytes of data:

Reply from 171.18.100.51: bytes=32 time=1ms TTL=128
Reply from 171.18.100.51: bytes=32 time=11ms TTL=128
Reply from 171.18.100.51: bytes=32 time<1ms TTL=128

Ping statistics for 171.18.100.51:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 11ms, Average = 4ms

Control-C
^C
C:\>
```

Now that we have a functional setup, we need to configure the VLANs based on the colors determined in the guide and the created diagram. In this case, VLAN 50 was assigned to the green ones, VLAN 55 to the purple ones, and VLAN 60, resulting in the following virtual network zones.

- Naranja:



- Verde:

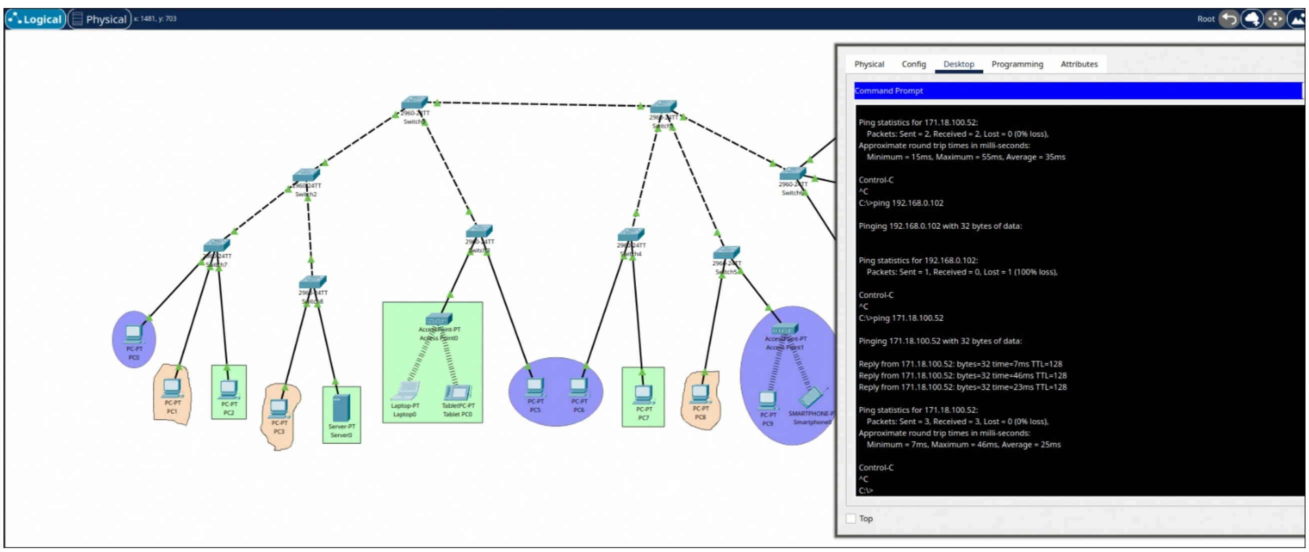
Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	In Progress	Smartph...	PC7	ICMP		0.000	N	0	(edit)	(delete)
	In Progress	PC7	Server0	ICMP		0.000	N	1	(edit)	(delete)
	In Progress	Server0	PC2	ICMP		0.000	N	2	(edit)	(delete)
	In Progress	PC2	PC7	ICMP		0.000	N	3	(edit)	(delete)
	In Progress	Server2	Server0	ICMP		0.000	N	4	(edit)	(delete)
	In Progress	Tablet PC1	Tablet PC0	ICMP		0.000	N	5	(edit)	(delete)
	In Progress	Laptop1	Server1	ICMP		0.000	N	6	(edit)	(delete)
	In Progress	Server0	Server1	ICMP		0.000	N	7	(edit)	(delete)
	In Progress	PC2	Laptop0	ICMP		0.000	N	8	(edit)	(delete)

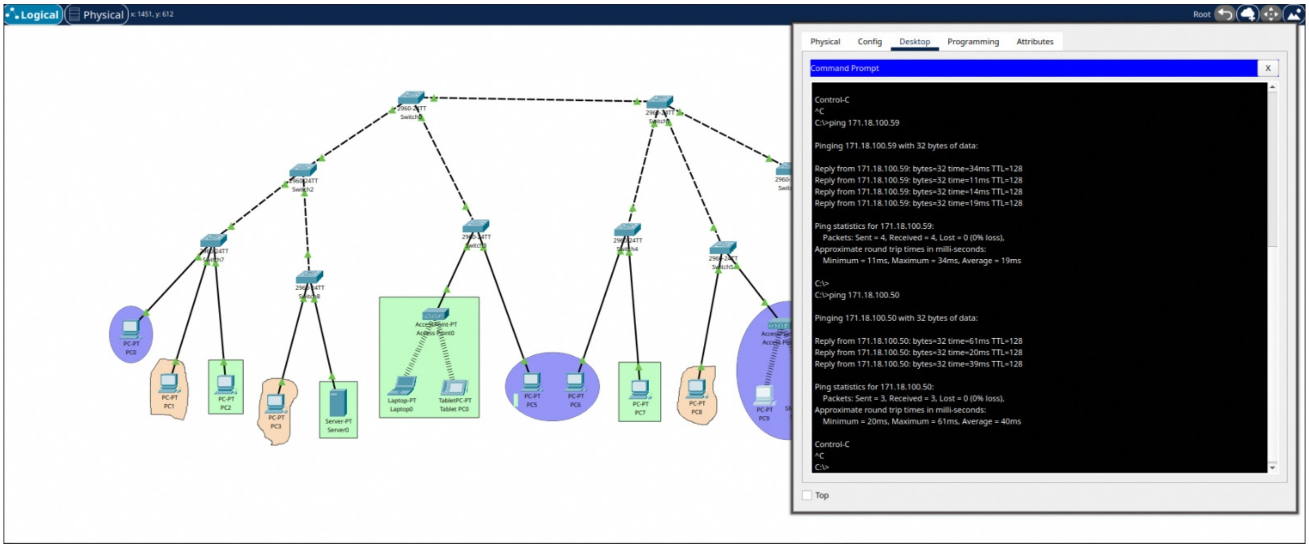
- Morado:

PDU List Window

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	In Progress	PC0	PC9	ICMP		0.000	N	0	(edit)	(delete)
	In Progress	PC0	Server1	ICMP		0.000	N	1	(edit)	(delete)
	In Progress	PC0	Smartphone0	ICMP		0.000	N	2	(edit)	(delete)
	In Progress	PC0	PC2	ICMP		0.000	N	3	(edit)	(delete)
	In Progress	PC0	PC5	ICMP		0.000	N	4	(edit)	(delete)
	In Progress	PC0	PC6	ICMP		0.000	N	5	(edit)	(delete)
	In Progress	PC0	PC7	ICMP		0.000	N	6	(edit)	(delete)
	In Progress	PC0	Server3	ICMP		0.000	N	7	(edit)	(delete)

Finally, we test by pinging between devices that are under the same VLAN, otherwise the connection is not recognized.



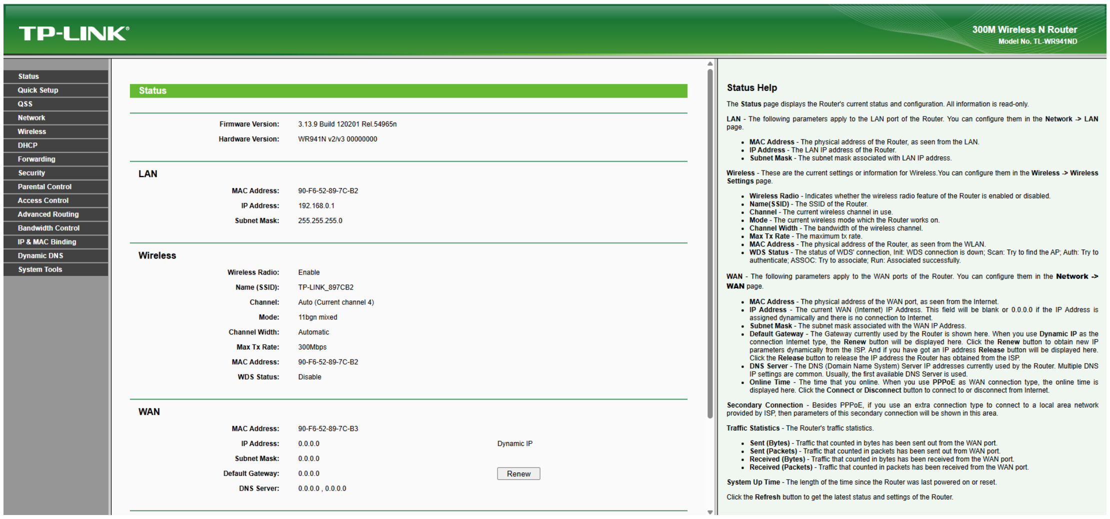


WIFI

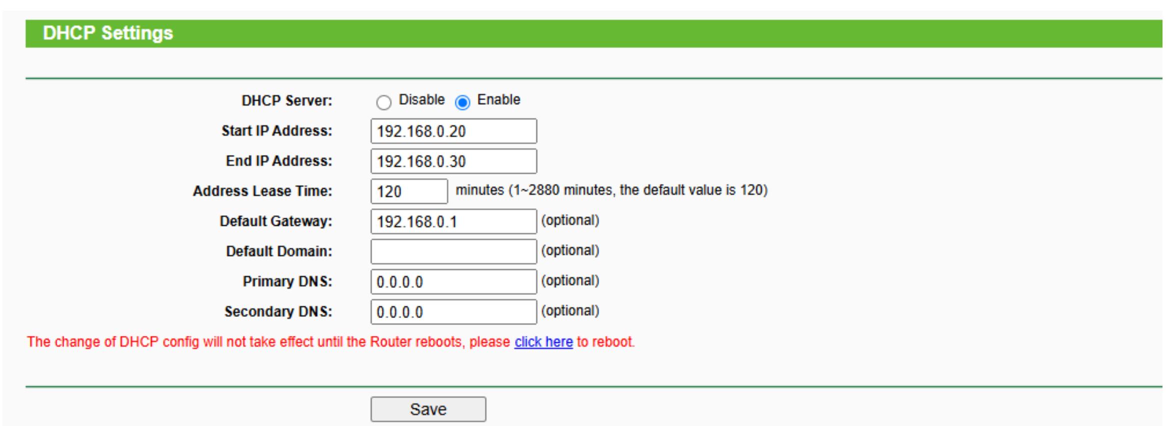
In this section, we have a physical router for wireless devices. The trick is to connect it, configure it, and validate its operation using our mobile device configuration. Initially, we just need to connect the router and turn it on. Then, we connect one leg to the university network with crossed Ethernet cables and the other to the computer we'll be using to configure it.

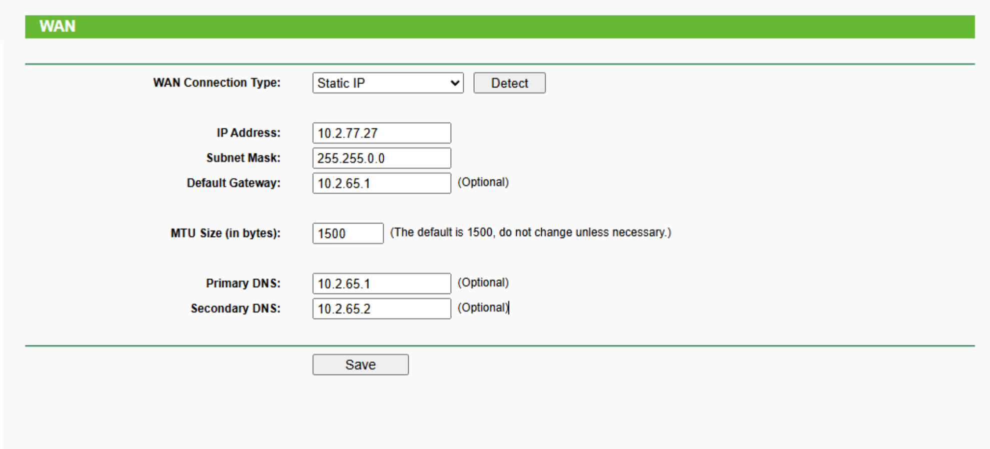
When we access the URL (tplinklogin.net) provided by the router manufacturer, we'll be asked for credentials, which are the admin username and password.

Once inside, we'll see a general dashboard containing all the information about the router and the network to which it's connected. We'll also see different tabs to manipulate these settings.

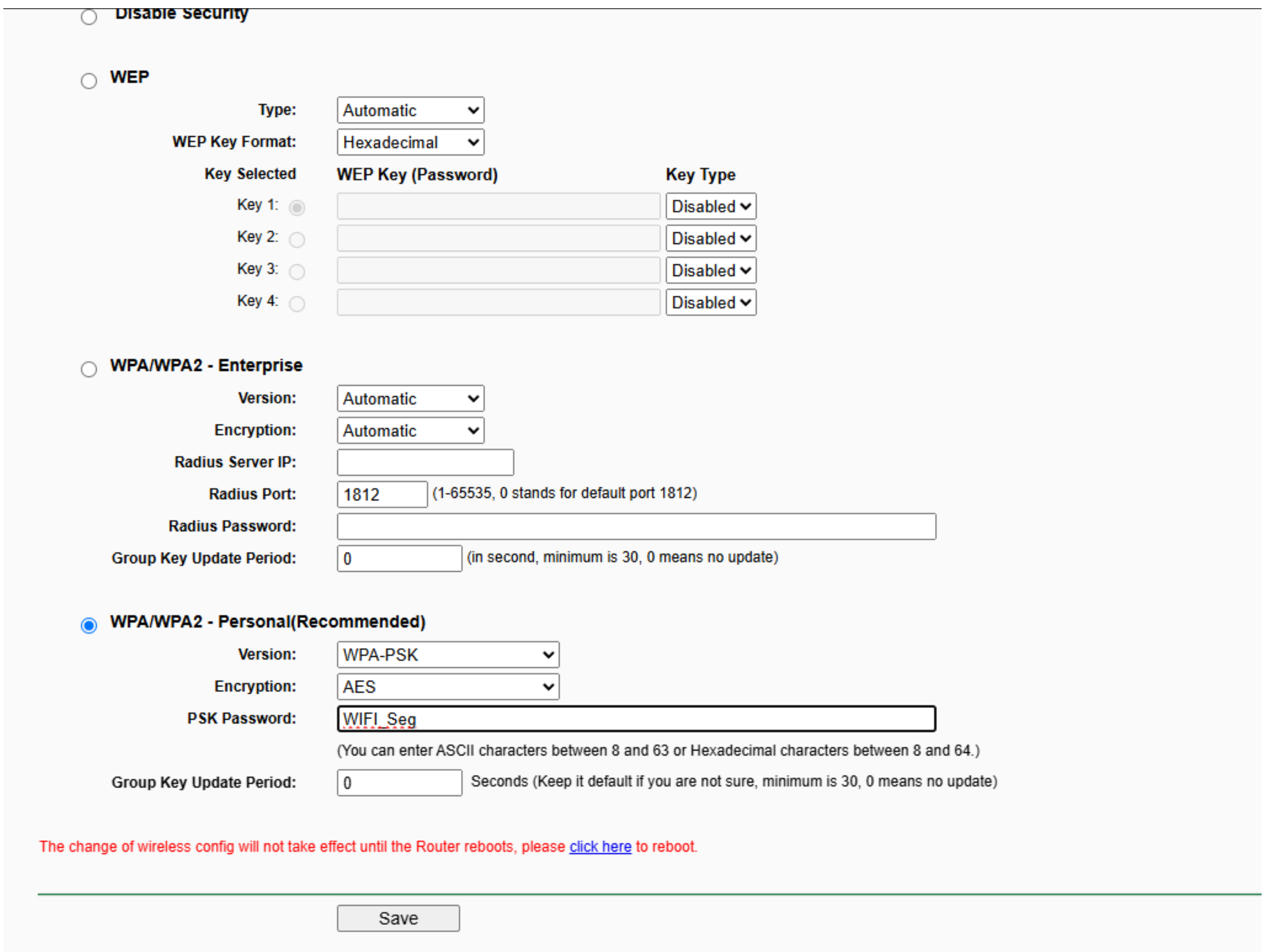


In our case, we're going to create a wireless network for our mobile devices. So, we'll access the WAN and DHCP settings, where we enter the necessary IP addresses assigned to our group.

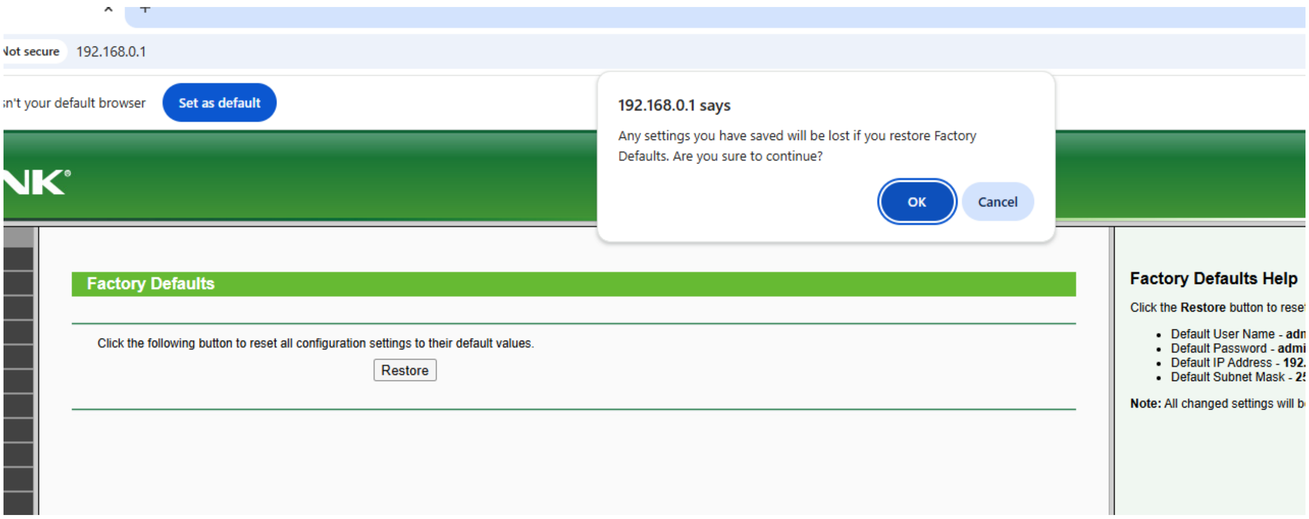




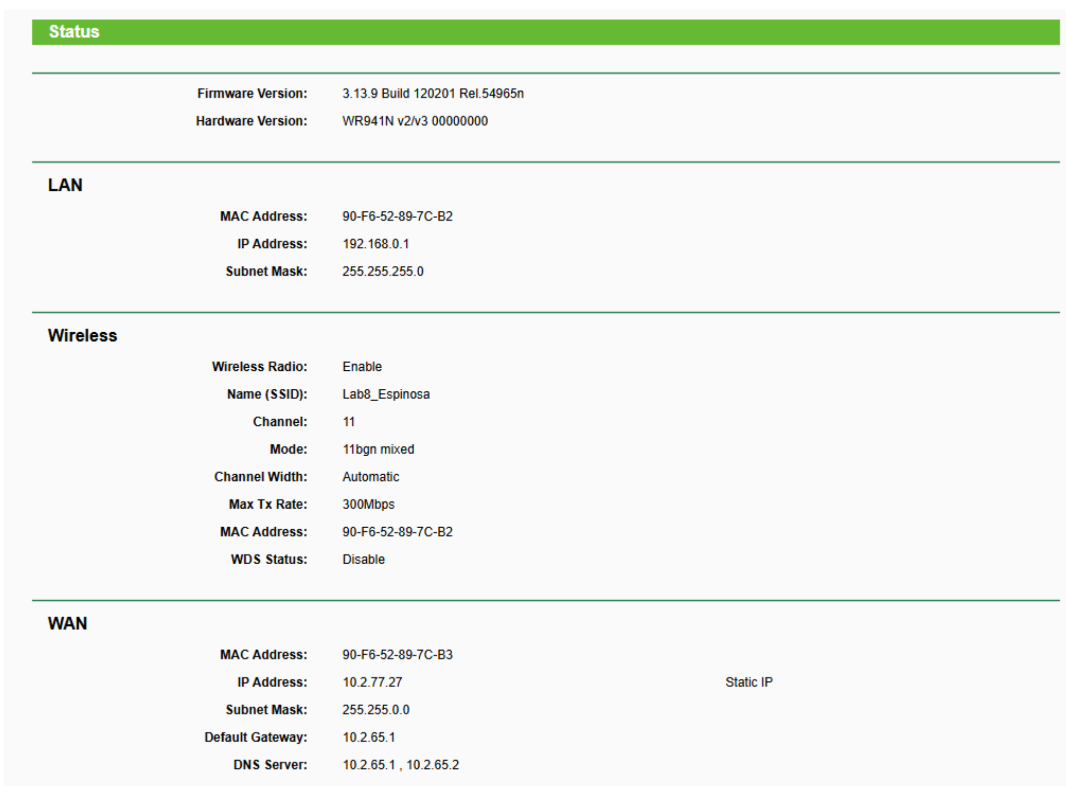
Now, the router is connected to the network. Therefore, it's necessary to configure the wireless network security by entering the password and WPA-PSK encryption and their respective settings.

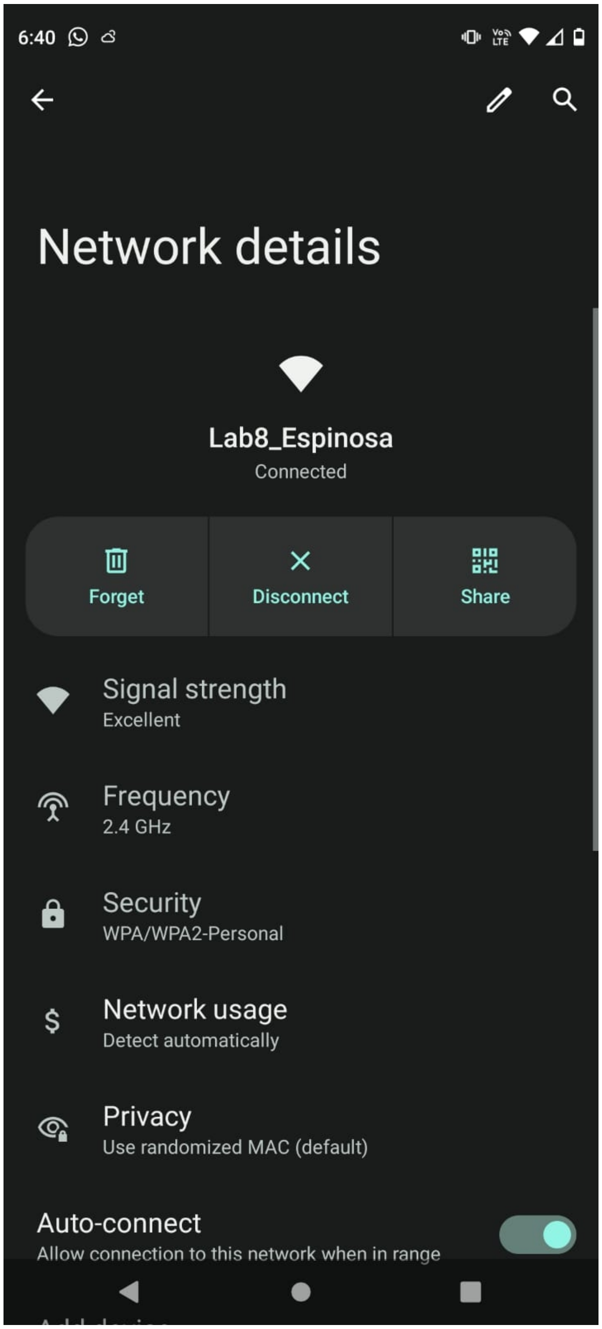


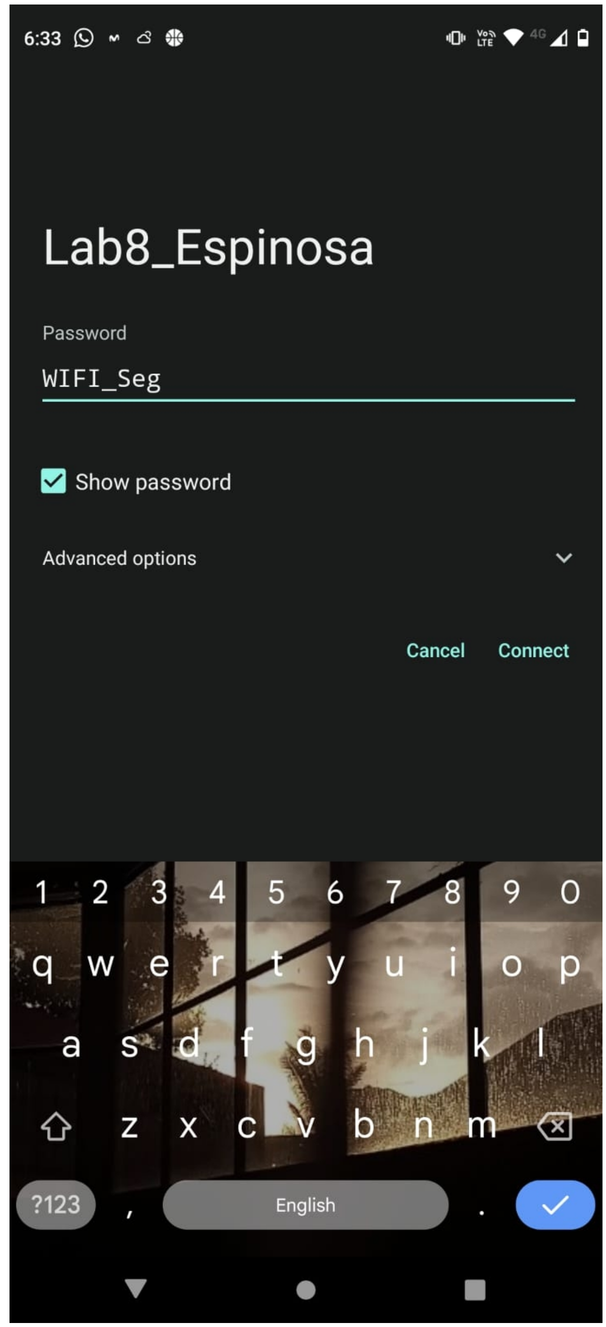
Now, we restart the router and confirm our action to make the configuration changes effective. This process takes a while, but once it's finished, we should have a functioning network.



Finally, we check the information about the deployed network and can see the created network on our mobile devices. This way, we can confirm that it's correctly finalized and ready to use.



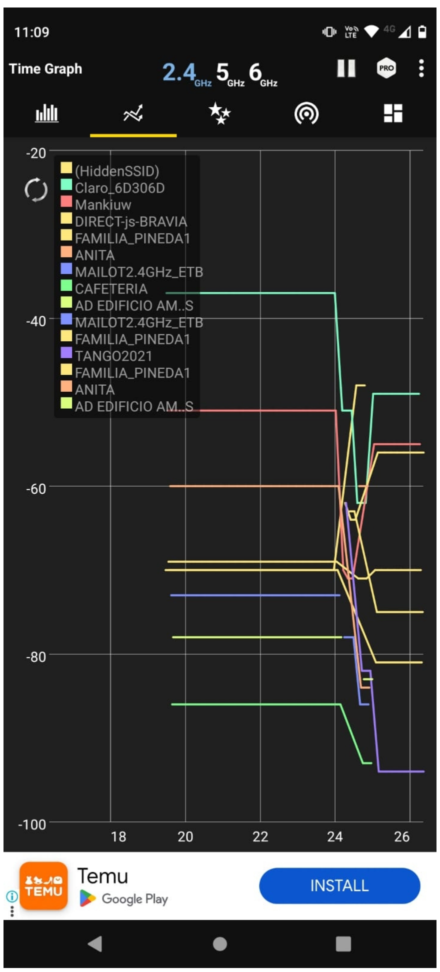

6:40
Network details
Lab8_Espinosa
Connected
Forget
Disconnect
Share
Signal strength
Excellent
Frequency
2.4 GHz
Security
WPA/WPA2-Personal
Network usage
Detect automatically
Privacy
Use randomized MAC (default)
Auto-connect
Allow connection to this network when in range

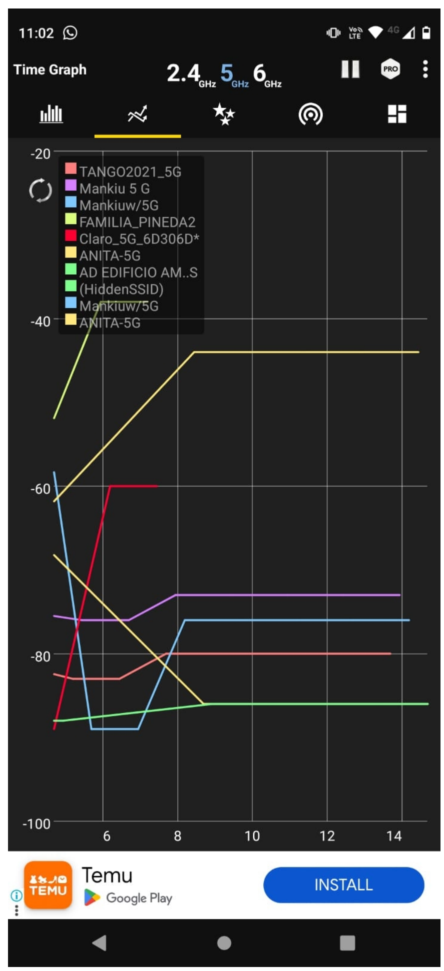

6:33
Lab8_Espinosa
Password
WIFI_Seg
Show password
Advanced options
Cancel
Connect
?123
English

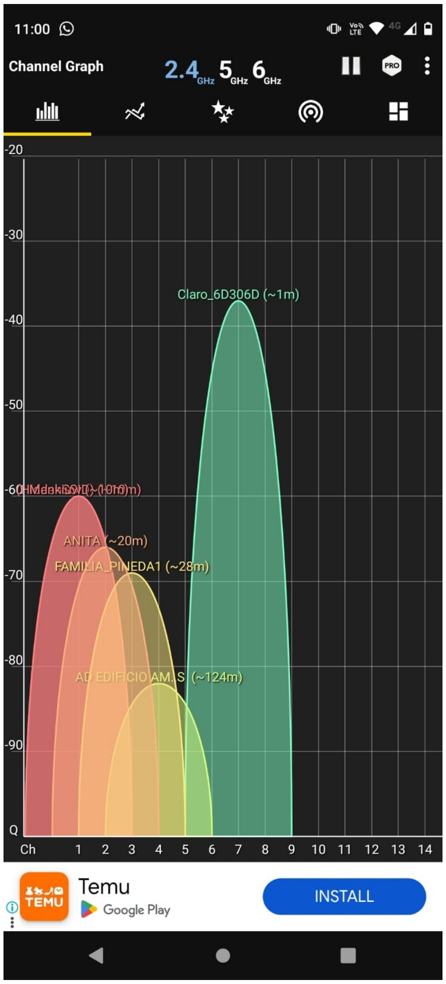
REVIEW WIFI NETWORKS NEAR YOUR HOME

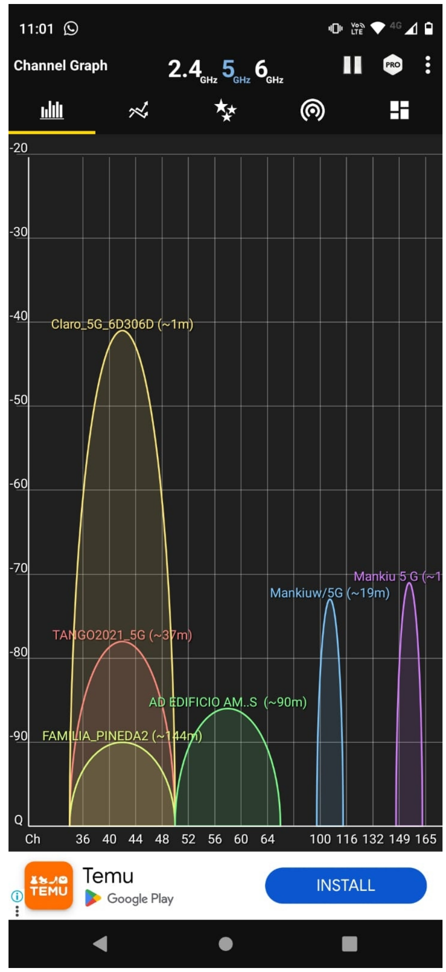
In this section of the lab, we are asked to use a wireless traffic monitoring app, such as WiFi Analyzer for Android, to explore the environment of nearby WiFi networks, including our own home network. The main objective is to identify available wireless networks in the area, collecting relevant information such as the network name, the frequency band in which they operate (2.4 GHz, 5.7 GHz, or 60 GHz), and the specific channels they use within those bands.

During the scan, a significant presence of networks operating in the 2.4 GHz band was found, although many of these had limited bandwidth and were subject to greater interference due to the common saturation of this band. On the other hand, networks operating in 5 GHz were the most predominant and prominent, as they offer better performance thanks to a greater number of available channels, less interference, and higher speeds in environments with good coverage.

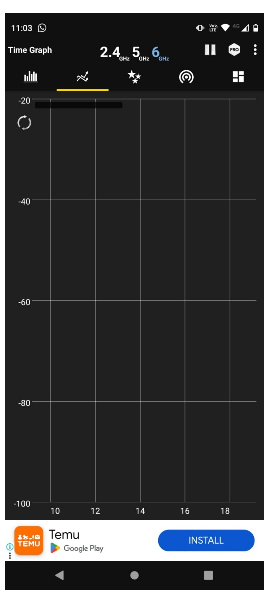


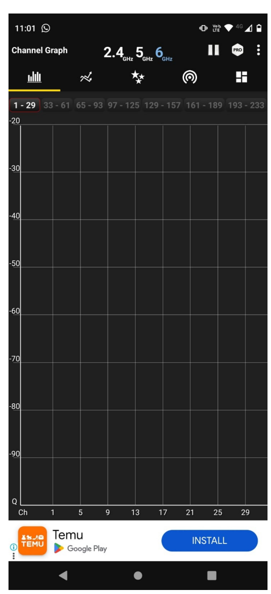






Regarding the 60 GHz band, no active networks were detected. This is expected, as this band is associated with newer technologies such as WiGig (802.11ad/ay), which are not yet widely deployed in home environments due to their high hardware requirements and limited coverage, given that 60 GHz waves do not easily pass through walls.





This exercise allowed us not only to identify the characteristics of the wireless environment, but also to understand how technologies vary depending on the operating band, coverage, speed, and level of adoption—critical aspects for the analysis and design of efficient Wi-Fi networks.

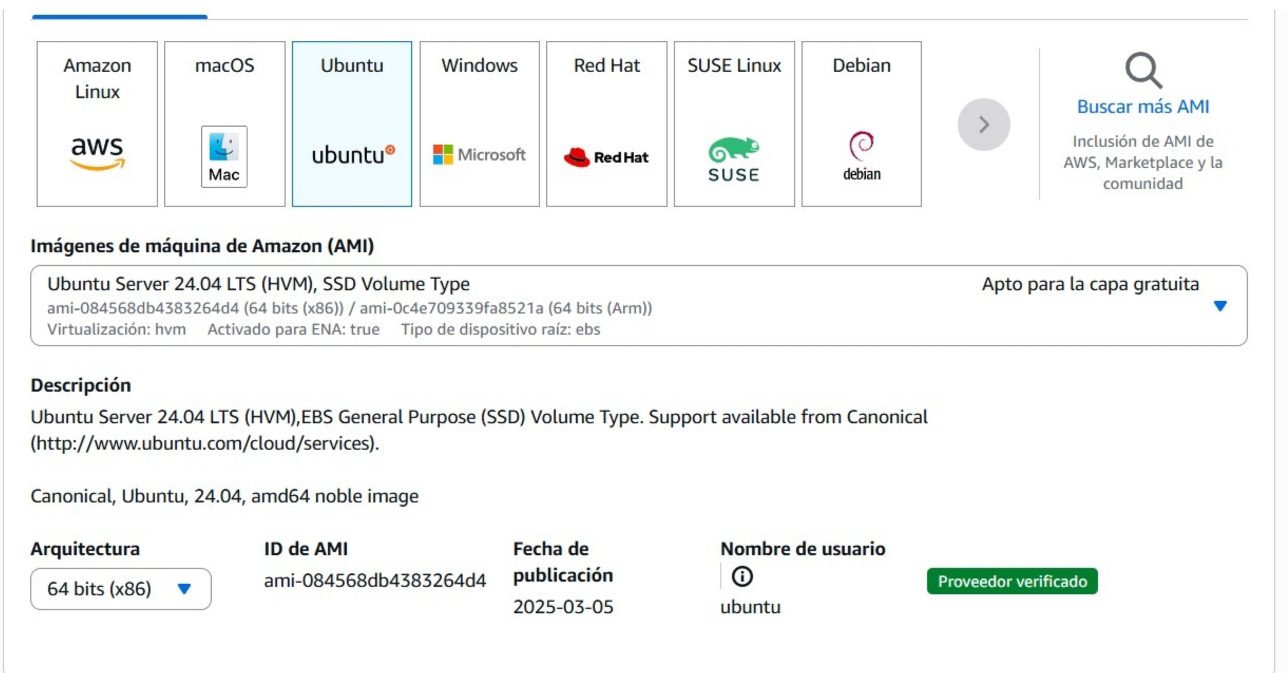
BASE SOFTWARE INSTALLATION

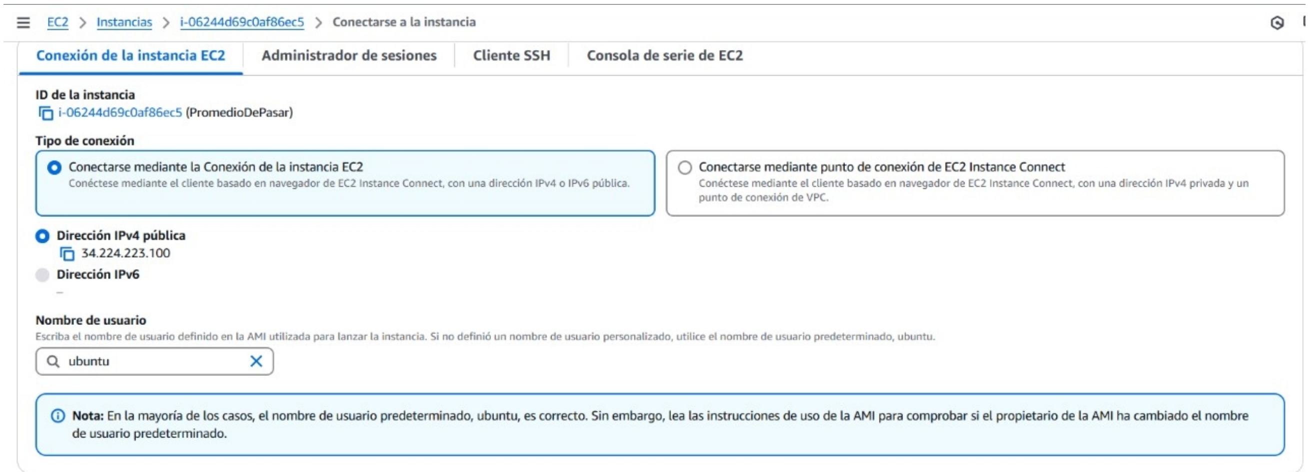
In this section, a dynamic web service is deployed using Apache and PHP on a cloud infrastructure. An application is developed to calculate final grades and store the data in a relational database, such as PostgreSQL. This practice allows you to apply application layer concepts and strengthen the interaction between web services and databases.

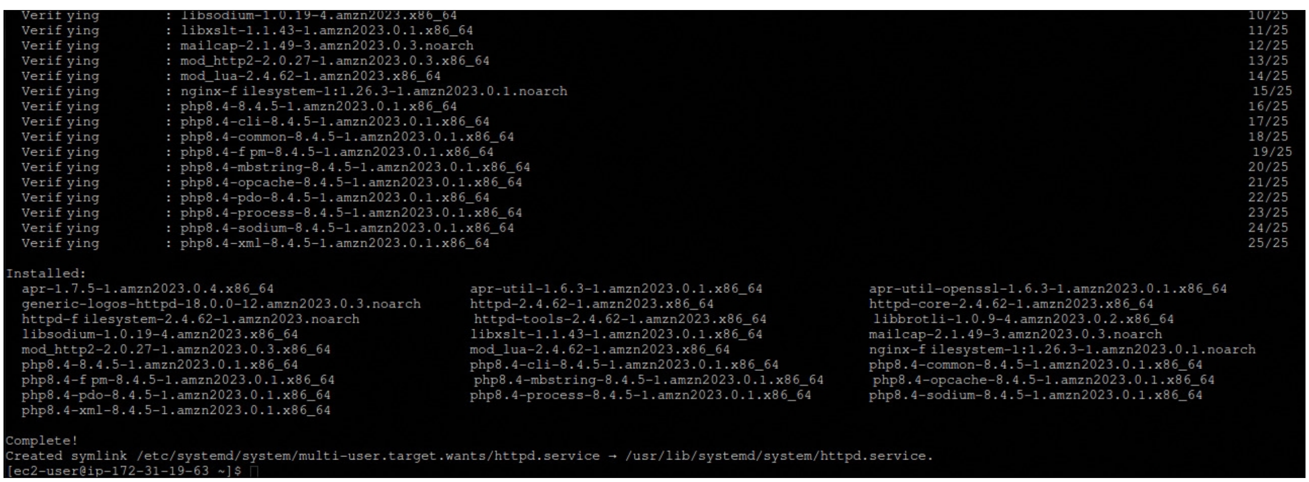
DYNAMIC WEB SERVICE

In this part of the lab, we focus on building and deploying a simple web application using an Ubuntu virtual machine hosted on AWS EC2. The main idea is to create a functional grade calculator that allows students to input their name and the grades for each third of the semester. The app then calculates the final grade based on a 30%, 30%, and 40% distribution, providing an immediate and clear result.

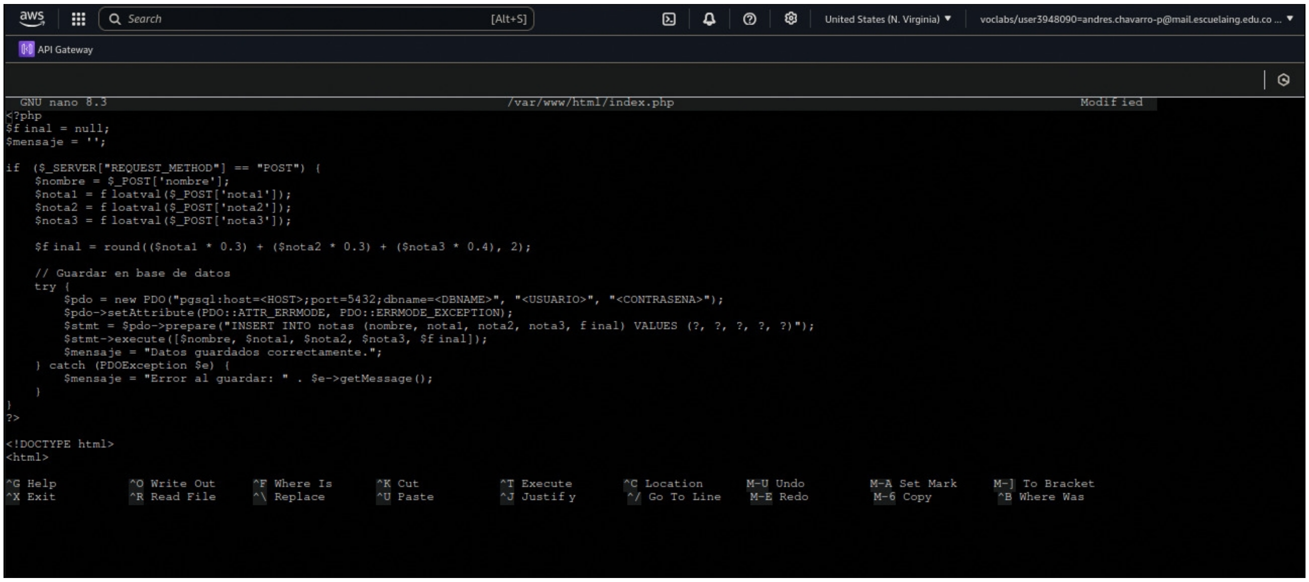
We begin by launching and configuring the EC2 instance. This includes updating the system, opening the necessary ports for HTTP traffic, and setting up the Apache web server to host our application. Once the server environment is ready, we install PHP and other required packages so the server can dynamically interpret and run our PHP-based webpage.





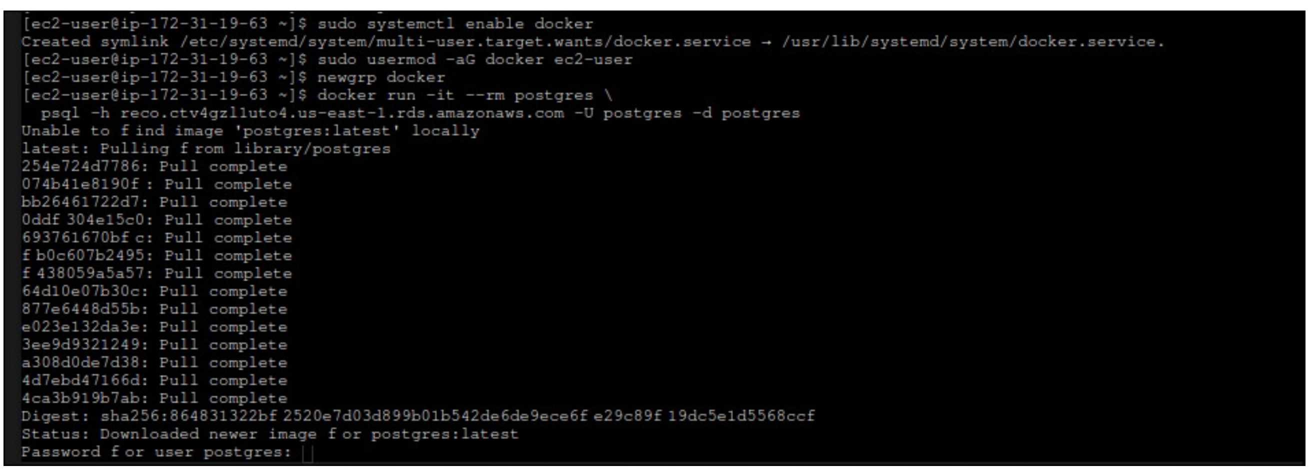


Next, we develop the application itself. The interface is straightforward: students can enter their name and three grades into a simple form. When the form is submitted, the PHP code processes the input, performs the weighted average calculation, and displays the final result directly on the webpage. This interaction makes the app both functional and easy to use.

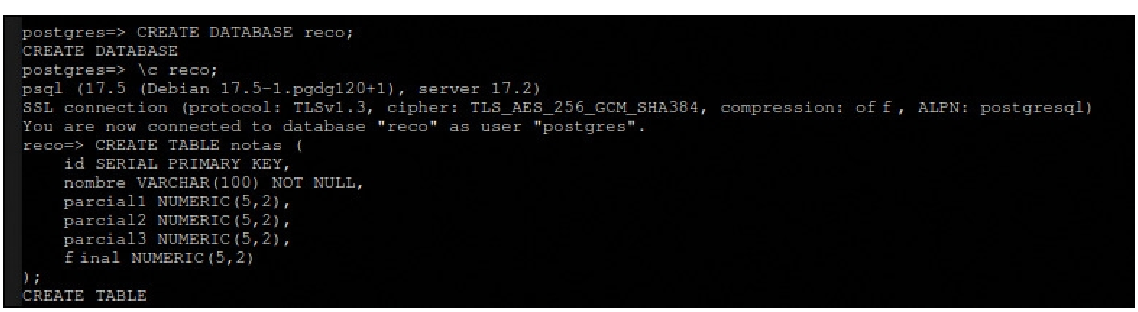


```
GNU nano 8.3                     /var/www/html/index.php                     Modified
<?php
$final = null;
$mensaje = '';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $nombre = $_POST['nombre'];
    $nota1 = floatval($_POST['nota1']);
    $nota2 = floatval($_POST['nota2']);
    $nota3 = floatval($_POST['nota3']);

    $final = round(($nota1 * 0.3) + ($nota2 * 0.3) + ($nota3 * 0.4), 2);

    // Guardar en base de datos
    try {
        $pdo = new PDO("pgsql:host=<HOST>;port=5432;dbname=<DBNAME>", "<USUARIO>", "<CONTRASENA>");
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        $stmt = $pdo->prepare("INSERT INTO notas (nombre, nota1, nota2, nota3, final) VALUES (?, ?, ?, ?, ?)");
        $stmt->execute([$nombre, $nota1, $nota2, $nota3, $final]);
        $mensaje = "Datos guardados correctamente.";
    } catch (PDOException $e) {
        $mensaje = "Error al guardar: " . $e->getMessage();
    }
}
?>

<!DOCTYPE html>
<html>

^G Help      ^O Write Out  ^F Where Is  ^K Cut    ^T Execute  ^C Location   M-U Undo  M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace   ^U Paste  ^J Justify  ^/ Go To Line M-E Redo  M-6 Copy      ^B Where Was
```

To store the submitted data, we integrate a relational database. We initially attempt to install PostgreSQL directly on the machine but face some complications. To work around this, we use Docker to containerize the PostgreSQL database, which simplifies management and improves compatibility. The Dockerized database connects smoothly with our application, allowing it to save and retrieve data effectively.

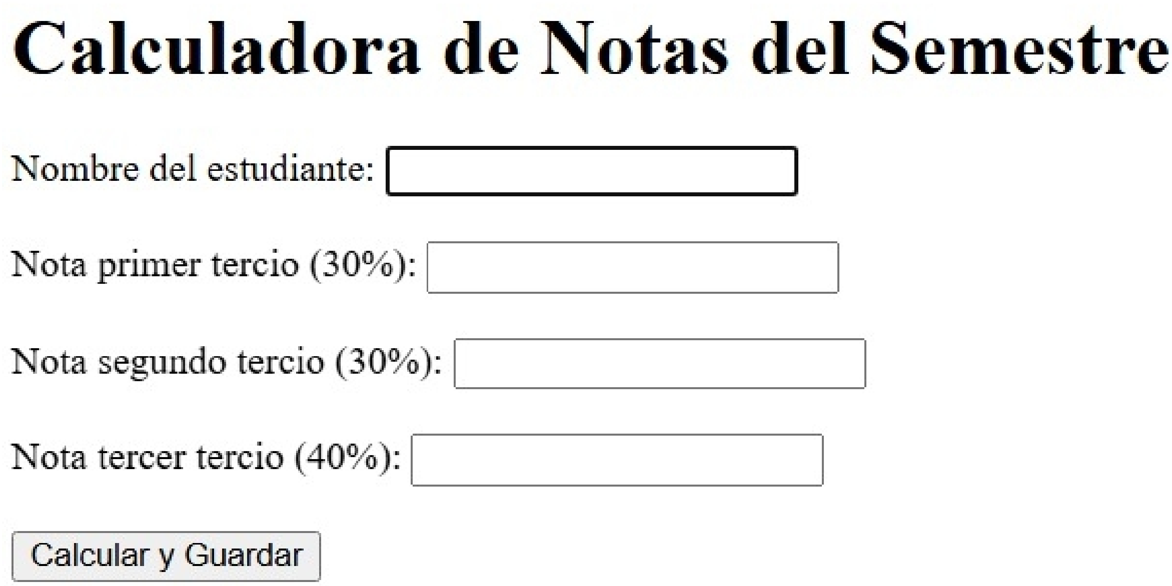


```
[ec2-user@ip-172-31-19-63 ~]$ sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[ec2-user@ip-172-31-19-63 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-172-31-19-63 ~]$ newgrp docker
[ec2-user@ip-172-31-19-63 ~]$ docker run -it --rm postgres \
  psql -h reco.ctv4gzlluto4.us-east-1.rds.amazonaws.com -U postgres -d postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
254e724d7786: Pull complete
074b41e8190f: Pull complete
bb26461722d7: Pull complete
0ddf304e15c0: Pull complete
693761670bfc: Pull complete
fb0c607b2495: Pull complete
f438059a5a57: Pull complete
64d10e07b30c: Pull complete
877e6448d55b: Pull complete
e023e132da3e: Pull complete
3ee9d9321249: Pull complete
a308d0de7d38: Pull complete
4d7ebd47166d: Pull complete
4ca3b919b7ab: Pull complete
Digest: sha256:864831322bf2520e7d03d899b01b542de6de9ece6fe29c89f19dc5e1d5568ccf
Status: Downloaded newer image for postgres:latest
Password for user postgres:
```



```
postgres=> CREATE DATABASE reco;
CREATE DATABASE
postgres=> \c reco;
psql (17.5 (Debian 17.5-1.pgdg120+1), server 17.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "reco" as user "postgres".
reco=> CREATE TABLE notas (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    parcial1 NUMERIC(5,2),
    parcial2 NUMERIC(5,2),
    parcial3 NUMERIC(5,2),
    final NUMERIC(5,2)
);
CREATE TABLE
```

After putting everything together, we test the application thoroughly. It performs well—the calculation logic is accurate, the data is successfully stored in the database, and the overall system runs reliably on the cloud. This experience gives us valuable insight into full-stack development, cloud deployment, and the practical use of tools like Apache, PHP, PostgreSQL, and Docker in a real-world scenario.



CONCLUTIONS

This lab allowed us to gain in-depth and practical knowledge of key network and application layer components, essential for the functioning of modern technological infrastructures. During the activities, we configured both wired and wireless networks, giving us a clearer understanding of how devices such as switches, routers, and access points operate, and how they interact at the data link layer to ensure effective data transmission.

One of the highlights was the configuration and analysis of Ethernet frames using Wireshark, which allowed us to study frame structure, MAC addresses, error control, and the process of frame learning and forwarding by switches. This gave us a detailed look at how switches handle network traffic at the data link layer, including the packet broadcasting process.

The implementation of VLANs was also critical, as it allowed us to segment the network into different broadcast domains, improving network security and performance. We effectively configured VLANs, assigning devices to different network segments, and set up trunking to allow traffic transmission between different switches and VLANs. This exercise reinforced our understanding of network segmentation concepts and the advantages of using VLANs in an enterprise infrastructure.

In the wireless networking segment, we configured a secure Wi-Fi network using WPA2-PSK with AES and learned how to manage mobile devices within this network. Analyzing nearby networks using applications such as Wi-Fi Analyzer also allowed us to explore channel management and security settings for wireless networks.

Furthermore, creating and configuring a dynamic web service, including integrating PHP with relational databases such as PostgreSQL, allowed us to better understand how web applications work and how they interact with database servers. This application layer component also involved implementing a shell script that executes and displays network commands, facilitating network administration and monitoring using tools such as ifconfig, netstat, vnstat, and route, among others.

In conclusion, this lab not only provided us with practical knowledge of network configuration and administration but also allowed us to understand the fundamental aspects of the interaction between the data link layer and the application layer, which is essential for designing, managing, and monitoring network infrastructures in the real world. This experience has given us a solid foundation for applying these concepts in professional situations, improving our ability to manage networks and services efficiently and securely.

BIBLIOGRAFY

Airheads community. (s/f). Airheads Community. Recuperado el 17 de mayo de 2025, de https://community.arubanetworks.com/browse/blogs/blogviewer?blogkey=cd567bc9-7c19-4561-bf80-c4470e5dc895

Configuring VLANs. (2015, marzo 21). Cisco. https://www.cisco.com/c/en/us/td/docs/switches/datacenter/sw/5_x/nx-os/layer2/configuration/guide/Cisco_Nexus_7000_Series_NX-OS_Layer_2_Switching_Configuration_Guide_Release_5-x_chapter4.html

Harmoush, E. (2016, agosto 11). *Configuring VLANs on Cisco switches*. Practical Networking .Net; Practical Networking. https://www.practicalnetworking.net/stand-alone/configuring-vlans/

Install and configure Apache. (s/f). Ubuntu. Recuperado el 17 de mayo de 2025, de https://ubuntu.com/tutorials/install-and-configure-apache

NetCloud Engineering. (2018). *Network engineering, telecommunications and cybersecurity in Barcelona,es*. https://netcloudengineering.com/configuracion-vlan-cisco-switch/?lang=en

Wireless RF reference guide. (2024, abril 19). Cisco. https://www.cisco.com/c/en/us/td/docs/wireless/controller/9800/technical-reference/wireless-rf-reference-guide.html

(S/f-a). Amazon.com. Recuperado el 17 de mayo de 2025, de https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html

(S/f-b). Amazon.com. Recuperado el 17 de mayo de 2025, de https://docs.aws.amazon.com/ec2/

(S/f-c). Hpe.com. Recuperado el 17 de mayo de 2025, de https://arubanetworking.hpe.com/techdocs/ArubaOS_8.12.0_Web_Help/Content/arubaos-solutions/6ghz-radio/6ghz-conf-para.htm